Qualcomm Technologies, Inc.

SW6100/SW6100P eNPU Guide – Linux Android Wear/Linux Embedded

80-93264-1 AC

April 28, 2026

About Qualcomm

Qualcomm relentlessly innovates to deliver intelligent computing everywhere, helping the world tackle some of its most important challenges. Building on our 40 years of technology leadership in creating era-defining breakthroughs, we deliver a broad portfolio of solutions built with our leading-edge AI, high-performance, low-power computing, and unrivaled connectivity. Our Snapdragon® platforms power extraordinary consumer experiences, and our Qualcomm Dragonwing™ products empower businesses and industries to scale to new heights. Together with our ecosystem partners, we enable next-generation digital transformation to enrich lives, improve businesses, and advance societies. At Qualcomm, we are engineering human progress.

Contents

1 Get started with SW6100/SW6100P eNPU

This section describes how the embedded neural processing unit (eNPU) differs from the standard NPU. It helps you identify low-power AI use cases, such as always-on voice detection, motion sensing, and face detection, and determine when the eNPU is the right accelerator for always-on and latency-sensitive workloads.

The Snapdragon Wear Elite platform includes multiple hardware intellectual property (IP) cores for artificial intelligence (AI) acceleration. The AI models can run on the CPU, graphics processing unit (GPU), neural processing unit (NPU), and eNPU, a dedicated low-power hardware accelerator. This document explains the eNPU hardware, AI use cases suitable for the eNPU, the LPAI (low-power AI) subsystem, and the workflow to deploy AI use cases on the eNPU.

The following figure shows the end-to-end AI model deployment workflow on Qualcomm platforms. You can use this workflow to build AI models with popular frameworks such as ONNX, PyTorch, and TensorFlow, and to optimize and deploy those models across different hardware backends.

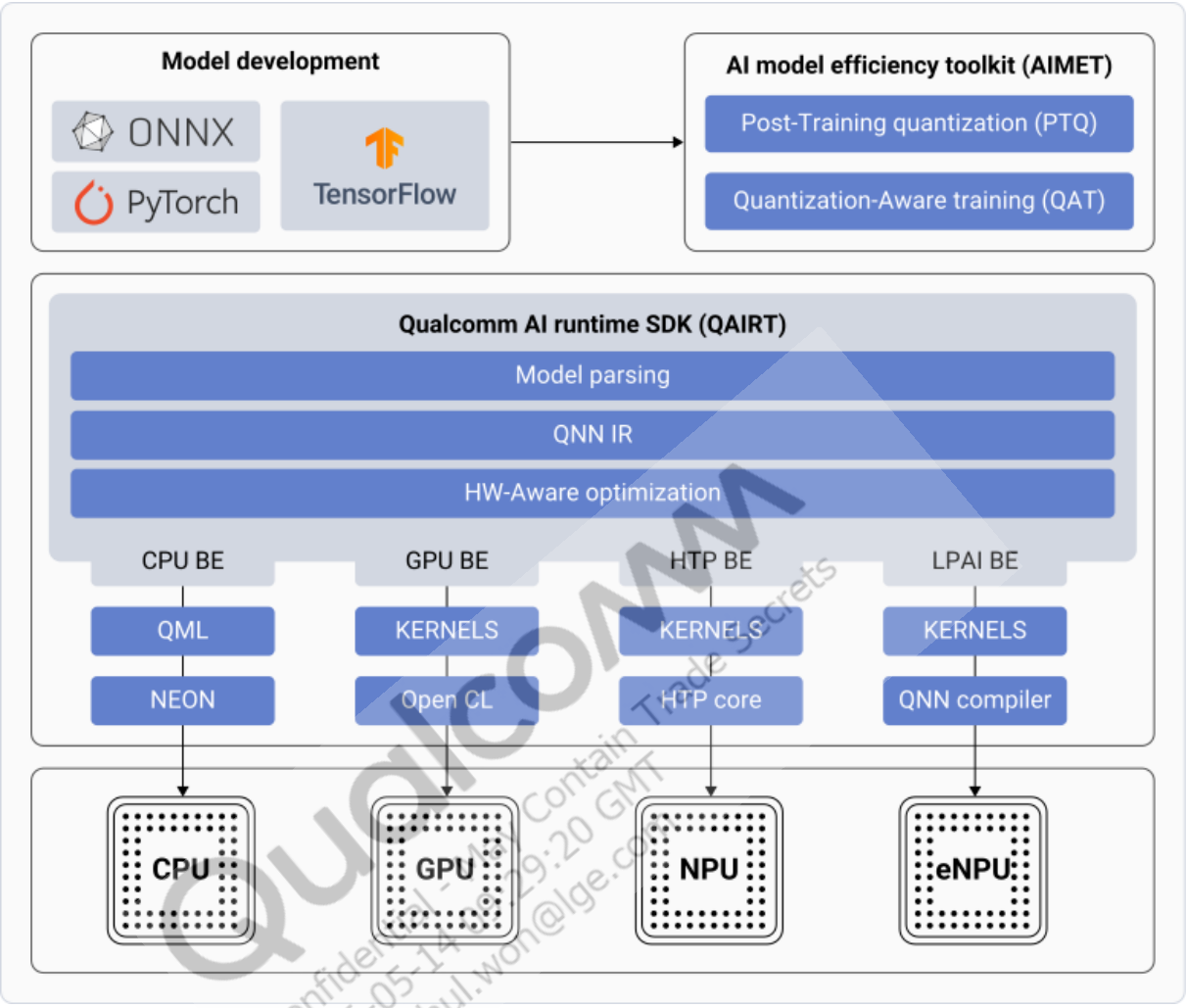


Figure : End-to-end AI model deployment flow on SW6100

- The Qualcomm® AI Runtime (QAIRT) SDK converts the AI models to Qualcomm optimized formats. The models then undergo model parsing, conversion to an intermediate representation (IR), and hardware-aware optimization.
- The platform maps the optimized model to appropriate backend cores such as CPU, GPU, Qualcomm® Hexagon™ Tensor Processor (HTP), or LPAI, based on kernel and compiler support. The model runs on the corresponding compute engine, such as the CPU, GPU, NPU, or eNPU to balance performance and power efficiency.
- Qualcomm also offers the AI model efficiency toolkit (AIMET) framework for post-training quantization or quantization-aware-training to improve accuracy of models.

The SW6100/SW6100P eNPU integrates with low-power subsystem (LPASS) or LPAI context to enable always-on, low-latency inference with minimal memory footprint.

The features of the eNPU are as follows:

- Has two variants, Big eNPU and Little eNPU to address complexities of various models or latency requirements
- Executes AI models within the low-power AI subsystem without explicit CPU initiation or processing
- Supports common neural operators, CNN and recurrent or attention-based models, using fixed-point inference (INT4/INT8/INT16)
- Supports compression features such as offline weight compression with runtime decompression
- Supports sparse activation compression

The eNPU accelerates its supported hardware layers but has strict limits on operator types, tensor dimensions, and layer parameters. Layers outside these limits may require padding, CPU preprocessing or postprocessing, or full CPU execution, reducing efficiency and increasing power use.

The following figure shows a top-level overview of the eNPU subsystem and its interaction with other system components, such as the Hexagon DSP. For more information about the eNPU subsystem architecture and its interaction with sensors and LPASS, see SW6100/SW6100P eNPU architecture.

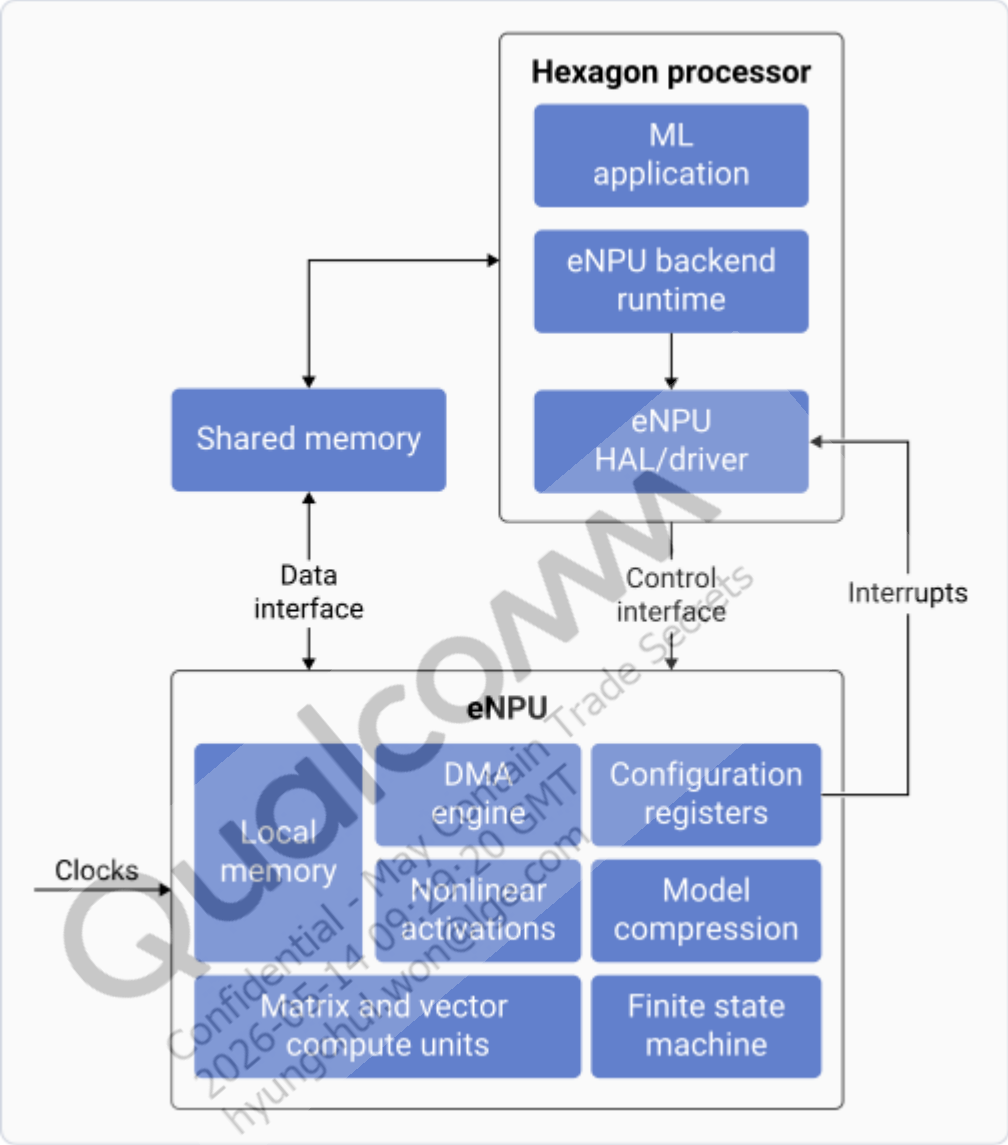


Figure : eNPU subsystem overview

The eNPU architecture together with the SW6100/SW6100P LPAI design makes eNPU well suited for low-power and low-latency inference workloads including the following categories:

- Always-on use cases, such as voice identification, speaker recognition, and speaker separation
- Sensing use cases, including motion sensing, activity recognition, and gesture recognition
- Vision-based detection use cases, such as face detection and gaze detection

1.1 How eNPU differs from NPU

On the Qualcomm SW6100/SW6100P platform, the eNPU and the NPU address different classes of AI workloads and power envelopes. The eNPU is a dedicated, always-on inference accelerator integrated within the LPAI subsystem, optimized for low-latency and low-power operation while the application processor remains idle.

In contrast, the SW6100/SW6100P NPU uses a Hexagon-based tensor processing architecture that combines a V81 scalar core with multiple Hexagon vector eXtensions (HVX), vector coprocessors, and Hexagon matrix eXtensions (HMX) matrix coprocessor, along with a vector tightly coupled memory (VTCM).

From a software perspective, the eNPU is typically accessed through low-power AI execution flows based on the QAIRT SDK, while NPU provides a more general-purpose tensor processing capability leveraging explicit scalar, vector, and matrix compute resources.

1.2 AI use case classification

LPAI supports multiple operating modes to balance power consumption and performance depending on system requirements:

- Ultra-low-power mode (ULPM): Keeps the eNPU in a deep low-power state with no computation, suitable for idle or always-on scenarios
- Low-power mode (LPM): Maintains partial readiness at reduced power for faster wake-up
- Active mode: Powers the eNPU to run QAIRT SDK models with maximum performance

The following table maps AI workloads on the SW6100/SW6100P platform to compute domains based on power, performance, and complexity, distinguishing workloads suited for low-power accelerators such as the eNPU from those requiring higher-performance compute engines.

Table : AI uses cases for compute domains

Attribute	eNPU: Always-on	eNPU: Low-power and low-latency	NPU: High performance
Primary intent	Ultra-low-power, always available detection and lightweight inference	Low latency inference within a constrained power envelope	Compute intensive AI requiring higher throughput and larger models
Voice and audio use cases	Voice activation (Stage 1) Voice UI Noise suppression Audio context	Voice call processing using echo cancellation noise suppression (ECNS) Voice UI (Stage 2)	Natural language processing Translation Transcription

Attribute	eNPU: Always-on	eNPU: Low-power and low-latency	NPU: High performance
Camera use cases	Always-on camera triggers Face detection Presence detection	Not applicable	Image manipulation Style transfer Video transformers Low light enhancement Super resolution Bokeh Segmentation
Sensors and Qualcomm sensing hub (QSH) use cases	Activity recognition Ultrasound proximity	Presence detection Proximity detection	Not applicable
Typical model size	50 KB to 1 MB	200 MB	10 MB to 500 MB
Typical compute requirement	0.1 GOPS/s to 5 GOPS/s	0.1 GOPS/s to 50 GOPS/s	50 GOPS/s to 1 TOPS/s

2 SW6100/SW6100P eNPU architecture

This section explains the embedded neural processing unit (eNPU) subsystem hardware and how it integrates with the low-power AI (LPAI) DSP to enable always-on inference. It describes the architectural components, data flow, and interactions that support low latency, power-efficient AI execution.

The Hexagon low-power AI DSP LPAIDSP controls the eNPU on SW6100/SW6100P platform through the Qualcomm AI Runtime (QAIRT) SDK. The eNPU accelerates quantized neural network inference using on-chip memory and a low-power interconnect. This inference enables always-on, low-latency AI execution while the rest of the SoC remains in a power-gated state.

The following figure shows a high-level architecture of the SW6100/SW6100P eNPU hardware:

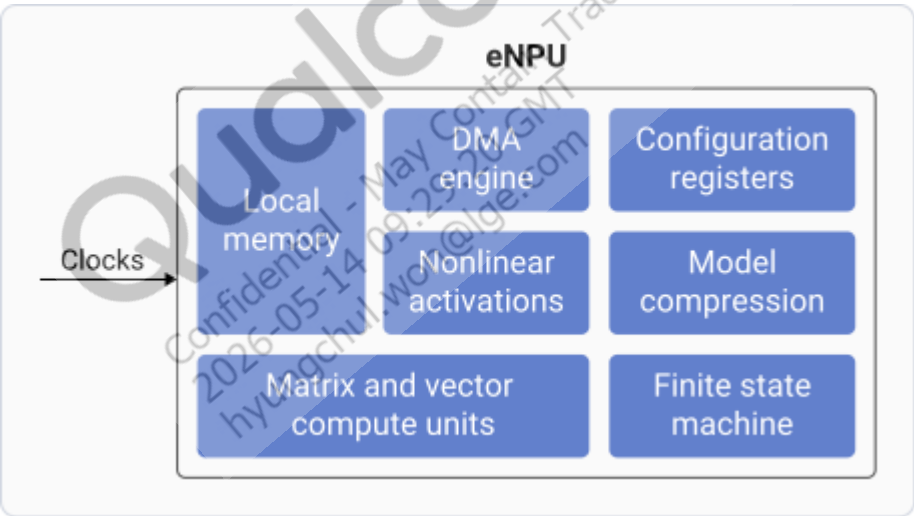


Figure : eNPU high-level hardware architecture

The SW6100/SW6100P eNPU hardware includes two main components: a sensor-wear microcontroller and a LPAIDSP. These components work together to manage sensor data and run neural network operations using Qualcomm frameworks and hardware accelerators.

- DMA engine: Automatically pulls data from shared memory into the local memory of the eNPU.
- Compute units: Dedicated hardware to perform matrix and vector math and nonlinear activation. For example, Sigmoid and ReLU.

The following figure shows the placement of eNPU with the Snapdragon Wear Elite platform low-power AI (LPAI) subsystem:

Figure : eNPU embedded in Snapdragon Wear Elite platform

The components of this architecture are:

- Sensor-wear microcontroller: This module manages sensor data and initial processing. It includes the Qualcomm sensing hub (QSH) 4.0, which aggregates sensor inputs and performs lightweight computations. The QAIRT SDK proxy acts as an intermediary that forwards the neural network requests to the DSP for execution. The DSP communicates through GLink, a high-speed inter-processor communication protocol.
- LPAIDSP: This is the core processing unit for advanced AI tasks. It also hosts QSH 4.0 to integrate sensors. The QAIRT SDK runs on the LPAIDSP and allows the software layer to run neural network models. The framework interfaces with the eNPU, which accelerates AI workloads. GLink connects this DSP to the microcontroller for seamless data exchange.
- GLink: A bidirectional communication channel that links the microcontroller and DSP, enabling fast and efficient data transfer for sensor and AI operations.
- eNPU: A specialized hardware accelerator designed for neural network computations to:
  - Reduce the processing burden on the DSP
  - Improve the performance for AI-driven features

3 Set up the host environment

This section explains how to set up your Linux or Windows host computer for embedded Neural Processing Unit (eNPU) development by configuring the required SDKs, tools, and dependencies. It guides you through setting up the Qualcomm® AI Runtime (QAIRT) SDK, model frameworks, and host prerequisites for reliable AI workflows.

The QAIRT SDK provides lower-level, unified APIs to develop AI models. The QAIRT SDK architecture provides components to build, optimize, and run network models on the best-suited core.

For more information about QAIRT SDK architecture, see Build AI models with QAIRT SDK.

3.1 Prerequisites

Complete the following prerequisites for any SW6100/SW6100P eNPU development task:

1. Set up a supported Linux or Windows host computer.
2. Download and install the QAIRT SDK.
3. Install all the required system dependencies and build tools for the host platform.
4. Create and activate a Python virtual environment.
5. Install required Python dependencies using the QAIRT SDK dependency checker.
6. Verify the environment using the QAIRT SDK validation utilities.
7. Install and configure Android Debug Bridge (ADB).
8. Install the Hexagon SDK.
9. Sign the audio digital signal processor (aDSP) libraries before pushing them from QAIRT SDK to the target device.

The following figure shows end-to-end Qualcomm Neural Network (QNN) workflow to develop AI models:

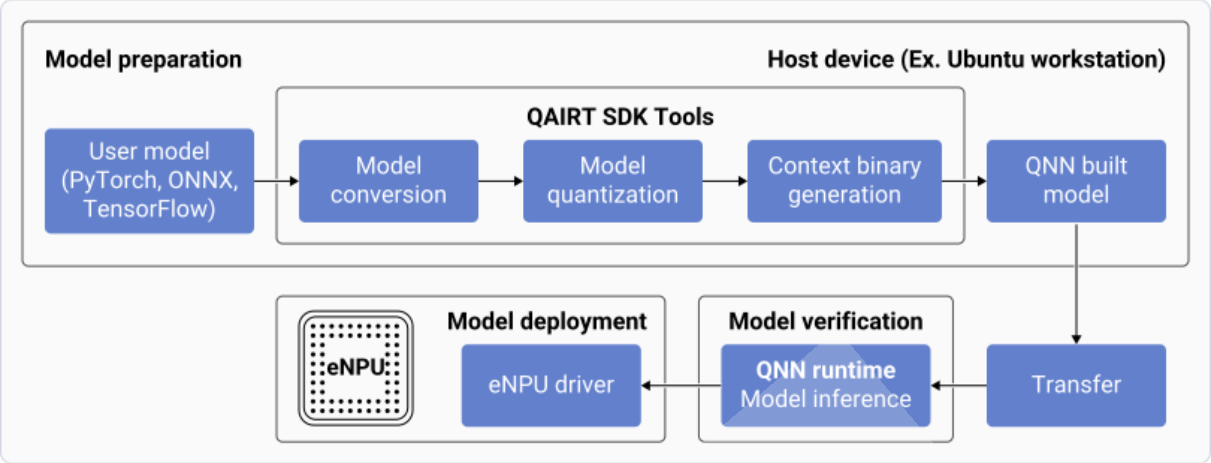


Figure : End-to-end QNN-based eNPU execution flows

3.2 Set up QAIRT SDK

To install the QAIRT SDK on your host computer, do the following:

1. Go to the Qualcomm AI Engine Direct SDK product page.
2. Select *Get Software* to download the QNN SDK.

> **Note:**
>
> The QNN SDK is approximately 2GB when unzipped.

3. Unzip the downloaded SDK.
   - The zip file has the version name, for example, `v2.22.6.240515`.
   - After extracting the files, rename the folder to `qairt`. This folder has a sub-folder with the version name, for example, `2.22.6.240515`.

3.3 Prepare the host environment

Set up the host to install and configure the downloaded QAIRT SDK. A correctly configured QAIRT SDK environment ensures reliable model conversion, profiling, execution, and provides smooth communication with the eNPU accelerator. The following sections provide step-by-step instructions to set up both Linux and Windows host computers, ensuring that each platform is correctly configured for QAIRT SDK installation and SW6100/SW6100P eNPU development.

3.4 Configure Linux host environment

Configure the Linux host computer according to the following requirements to ensure that it's ready for SW6100/SW6100P eNPU development and supports QNN-based workflows.

3.4.1 Install QNN SDK

1. Open a Linux terminal.
2. Go to `qairt/<SDK_VERSION>` in the unzipped QNN SDK directory.
   Replace the `<SDK_VERSION>` with the name of the file located directly under `qairt`. For example, `2.22.6.240515`.
3. Set up the `QAIRT_SDK_ROOT` environment variable to point to the required SDK using the following commands:

```
cd bin
source ./envsetup.sh
```

> **Warning:**
>
> The `envsetup.sh` script updates environment variables only for the current terminal session. If you need to set `QAIRT_SDK_ROOT` again, re-run the script. Alternatively, you can persist the environment variables configured by the script.

> **Warning:**
>
> The `QNN_SDK_ROOT` and `SNPE_ROOT` environment variables are currently set to `QAIRT_SDK_ROOT` for backward compatibility. Future releases support only the `QAIRT_SDK_ROOT` variable. Update any relevant workflows to use `QAIRT_SDK_ROOT` instead of `QNN_SDK_ROOT` or `SNPE_ROOT`.

4. Run the following command to install the Linux build tools:

```
sudo ${QAIRT_SDK_ROOT}/bin/check-linux-dependency.sh
```

   After all required dependencies are installed, the script displays the message "`Done!!`".

> **Note:**
>
> While the script is running, confirm any additional downloads by selecting *Enter.* The script may take several minutes to complete.

5. Verify that you have installed the required toolchain successfully by running the following command:

```
${QAIRT_SDK_ROOT}/bin/envcheck -c
```

```
(MY_ENV_NAME) :/local/mnt/workspace/mohanga/tmp_test_enpu/qairt/2.42.0.251225/bin$ ${QAIRT_SDK_ROOT}/bin/envcheck -c
Checking Clang Environment
---------------------------------------------------------
[INFO] Found clang++ at /usr/bin/clang++
---------------------------------------------------------
```

3.4.2 Install QNN SDK dependencies

1. Install Python 3.10 using the following command:

```
sudo apt-get update && sudo apt-get install python3.10 python3-distutils
libpython3.10
```

2. Verify if the installation worked by running:

```
python3 --version
```

> **Warning:**
>
> The QNN SDK is verified with Python 3.10. Ensure that you have installed Python 3.10.

3. Run the `cd ${QAIRT_SDK_ROOT}` command.
4. Install `python3.10-venv` if you don't have it installed already by running:

```
sudo apt install python3.10-venv
```

5. Run the following commands to create and activate a new virtual environment (you may rename `MY_ENV_NAME` according to your preference):

```
python3 -m venv MY_ENV_NAME --without-pip
source MY_ENV_NAME/bin/activate
python3 -m ensurepip --upgrade
```

6. Run `which pip3` to verify that the virtual environment has a local version of `pip3`. You should see a path that's inside your virtual environment. For example, `/MY_ENV_NAME/bin/pip3`.
7. Update all dependencies by running the following command:

```
python "${QAIRT_SDK_ROOT}/bin/check-python-dependency"
```

> **Warning:**
>
> If you face an error with a specific package version, run `pip install PACKAGE_NAME` to install a more up-to-date version of the package, and then re-run the script.

> **Note:**
>
> This command installs all required packages. To install optional packages, pass `-o|--with-optional`.

3.4.3 Install model frameworks

QNN supports the following model frameworks. Install only the frameworks required for your AI model files.

> **Warning:**
>
> You don't need to install all of the listed packages.

> **Note:**
>
> To install a specific package, use the `pip3 install package==version` command. For example: `pip3 install torch==1.13.1`.

Table : Supported model framework packages

Package	Version	Description
torch (SNPE/QNN)	1.13.1	PyTorch uses `.pt` files to build and train the deep learning models with a focus on flexibility and speed. Download and install the correct binary from PyTorch previous versions if pip install doesn't work.
torch (QAIRT)	2.4.1	PyTorch uses `.pt` files to build and train deep learning models with a focus on flexibility and speed. Download and install the correct binary from PyTorch previous versions if pip install doesn't work.
torchvision	0.14.1 (SNPE/QNN) / 0.19.1 (QAIRT)	Torchvision runs computer vision tasks with PyTorch, providing datasets, model architectures, and image transforms.
tensorflow	2.10.1	TensorFlow uses `.pb` files to build and train machine learning models, particularly deep learning models. **Note:** The `envcheck` script incorrectly reports that this file isn't installed on Ubuntu.
tflite	2.18.0	TFLite uses `.tflite` files and runs TensorFlow models on mobile and edge devices with optimized performance.
onnx	1.17.0	Open Neural Network Exchange (ONNX) uses `.onnx` files to define and exchange deep learning models between different frameworks.
onnxruntime	For Ubuntu 22.04 - 1.22.0 For Ubuntu 20.04 - 1.19.2	Open Neural Network Exchange (ONNX) uses `.onnx` files to run ONNX models with high performance across various hardware platforms.
onnxsim	0.4.36	Onnxsim uses `.onnx` files and simplifies ONNX models to reduce complexity and improve inference efficiency.

> **Note:**
>
> You can run the `${QAIRT_SDK_ROOT}/bin/envcheck -a` command to verify that you've installed the required dependencies.

3.4.4 Install dependencies for target hardware processors

Some target processors such as CPU, GPU, HTP, compute DSP (cDSP), or Qualcomm® Hexagon™ Tensor Accelerator (HTA) require additional dependencies to build models.

CPU dependencies

The CPU uses the `clang-14` compiler to build the `x86_64` targets. If working with similar targets, install `clang++14` from low-level virtual machine compiler framework (LLVM): `Clang++14`.

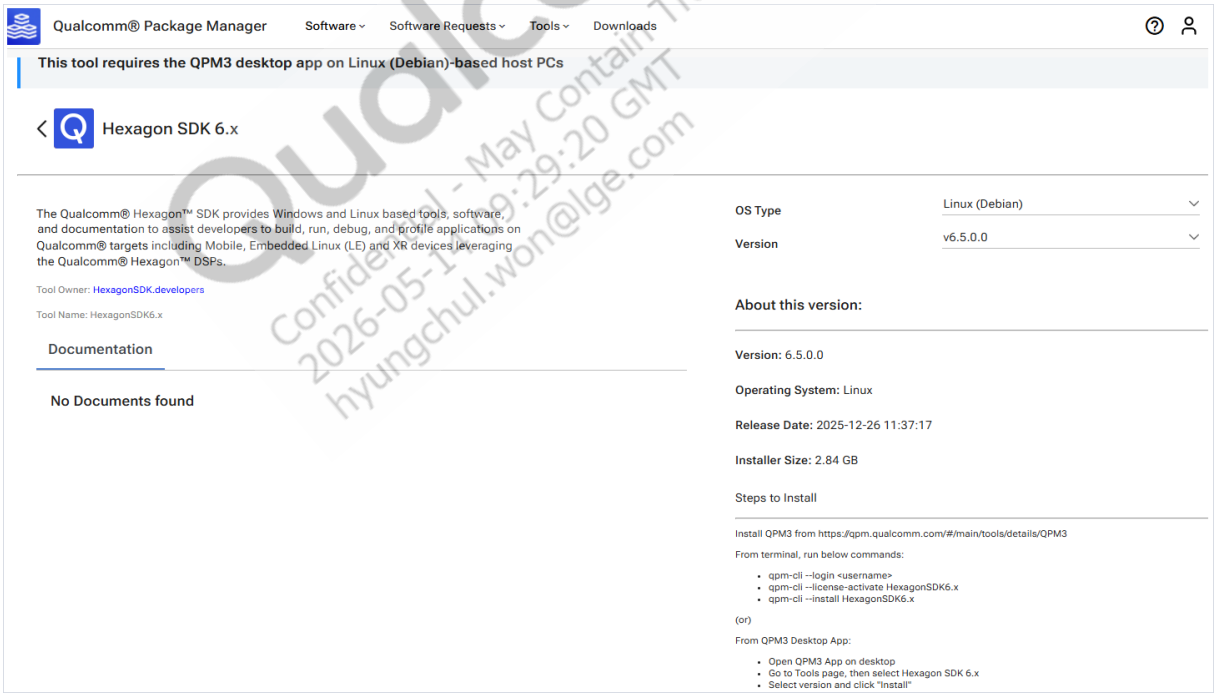
The Android NDK builds Arm CPU targets.

GPU dependencies

The GPU backend kernels use OpenCL. Implement the GPU operations based on the OpenCL headers. Ensure that you use OpenCL version 1.2 or later.

HTP and DSP dependencies

Compile both HTP and DSP hardware using the Qualcomm® Hexagon™ SDK and QNN SDK. You can download and install Hexagon SDK from the Qualcomm Package Manager (QPM).



LPAI dependencies

The LPAI backend supports offline model preparation only.

3.5 Configure Windows host environment

Configure the Windows host computer according to the following requirements to ensure it's ready for SW6100/SW6100P eNPU development and supports QNN-based workflows.

3.5.1 Install QNN SDK

1. To open an administrator PowerShell terminal do the following:
   a. Select the *Windows* option to search for applications.
   b. Search for *PowerShell*.
   c. Select *Windows PowerShell* followed by *Run as Administrator*.
   d. Select *Yes* when prompted for permission to run the application as an administrator.
2. Go to `qairt/<SDK_VERSION>` in the unzipped QNN SDK directory.
   Replace `<SDK_VERSION>` with the name of the file located directly under `qairt`. For example, `2.22.6.240515`.
3. Set up the `QAIRT_SDK_ROOT` environment variable using the following commands:

```
cd bin
Unblock-File ./envsetup.ps1
./envsetup.ps1
```

> **Warning:**
>
> The `envsetup.ps1` script updates environment variables only for the current PowerShell session. If you need to set `QAIRT_SDK_ROOT` again, rerun this script. Alternatively, you can persist the environment variables configured by the script in your shell environment.

4. Run `Unblock-File "${QAIRT_SDK_ROOT}/bin/check-windows-dependency.ps1"`.
5. Run the following command to install the core Windows build tools:

```
& "${QAIRT_SDK_ROOT}/bin/check-windows-dependency.ps1"
```

   - Each time you run the script, it installs additional dependencies. You may have to re-run the command after it installs Visual Studio and Visual Studio build tools.
   - When you have installed all the necessary dependencies, you can see the message `"All Done"`.

6. Run `Unblock-File "${QAIRT_SDK_ROOT}/bin/envcheck.ps1"`.

7. Verify that you have installed the Microsoft Visual Studio C++ (MSVC) toolchain successfully by using the following command:

```
& "${QAIRT_SDK_ROOT}/bin/envcheck.ps1" -m
```

> **Note:**
>
> The system installs either the `Visual C++(x86)` or `Visual C++(arm64)` toolchain. Ensure that the installed version matches the hardware of your host computer.

The toolchain installation displays the following output:

```
Checking MSVC Toolchain
-------------------------------------------------------------
WARNING: The version of VS BuildTools 14.40.33807 found has not
been validated. Recommended to use known stable VS
BuildTools version 14.34
WARNING: The version of Visual C++(x64) 19.40.33812 found has not
been validated. Recommended to use known stable
Visual C++(x64) version 19.34
WARNING: The version of CMake 3.28.3 found has not been validated.
Recommended to use known stable CMake version 3.21
WARNING: The version of clang-cl 17.0.3 found has not been
validated. Recommended to use known stable clang-cl version
15.0.1
Name              Version
----              -------
Visual Studio     17.10.35027.167
VS Build Tools    14.40.33807
Visual C++(x86)   19.40.33812
Visual C++(arm64) Not Installed
Windows SDK       10.0.22621
CMake             3.28.3
clang-cl          17.0.3
-------------------------------------------------------------
```

3.5.2 Install QNN SDK dependencies

1. Install Python 3.10.4.

   > **Warning:**
   >
   > Ensure that you download the correct Python version. New releases beyond Python `3.10.4` may not work as expected with the QNN SDK.

2. Verify that the installation was successful by running:

   ```
   py --list
   ```

   The installation is successful if `-V:3.10` shows as one of the options in the output.

3. Open the `QAIRT_SDK_ROOT` folder in Visual Studio.
4. Open a new PowerShell terminal in Visual Studio. From the top-menu, select *Terminal* > *New Terminal*.

   > **Warning:**
   >
   > Ensure that the new terminal is a PowerShell instance.

5. From the `QAIRT_SDK_ROOT` folder, run the following command to create and activate a new virtual environment:

   ```
   py -3.10 -m venv "venv"
   & "venv\Scripts\Activate.ps1"
   ```

6. Update all dependencies by running the following commands in the PowerShell:

   ```
   python -m pip install --upgrade pip
   python "$env:QAIRT_SDK_ROOT\bin\check-python-dependency"
   ```

   > **Note:**
   >
   > This command installs all required packages. To install optional packages pass `o|--with-optional`.

3.5.3 Install model frameworks

QNN supports the following model frameworks. Install the model frameworks required for your AI model.

Warning:

You don't need to install all of the listed packages.

Note:

You can install a specific package by running the `pip install package==version` command. For example, `pip install torch==1.13.1`.

Table : Supported model framework packages

Package	Version	Description
torch (SNPE/QNN)	1.13.1	PyTorch uses `.pt` files to build and train the deep learning models with a focus on flexibility and speed. Download and install the correct binary from PyTorch previous versions if pip install doesn't work.
torch (QAIRT)	2.4.1	PyTorch `.pt` files to build and train the deep learning models with a focus on flexibility and speed. Download and install the correct binary from PyTorch previous versions if pip install doesn't work.
torchvision	0.14.1 (SNPE/QNN) or 0.19.1 (QAIRT)	Torchvision runs computer vision tasks with PyTorch, providing datasets, model architectures, and image transforms.
tensorflow	2.10.1	Uses `.pb` files to build and train machine learning models, particularly the deep learning models.
tflite	2.18.0	Uses `.tflite` files to run TensorFlow models on mobile and edge devices with optimized performance.
onnx	1.17.0	ONNX uses `.onnx` files to define and exchange deep learning models between different frameworks.
onnxruntime	For Ubuntu 22.04: 1.22.0 For Ubuntu 20.04: 1.19.2	ONNX runtime uses `.onnx` files to run ONNX models with high performance across various hardware platforms.
onnxsim	0.4.36	Uses `.onnx` files to simplify ONNX models, reduce complexity, and improve inference efficiency.

Note:

You can run the `& "${QAIRT_SDK_ROOT}/bin/envcheck.ps1" -a` command to verify the installation and check that you've installed the required dependencies.

3.5.4 Install dependencies for target hardware processors

Some of the target processors such as CPU, GPU, HTP or HTA require additional dependencies to build models.

CPU dependencies

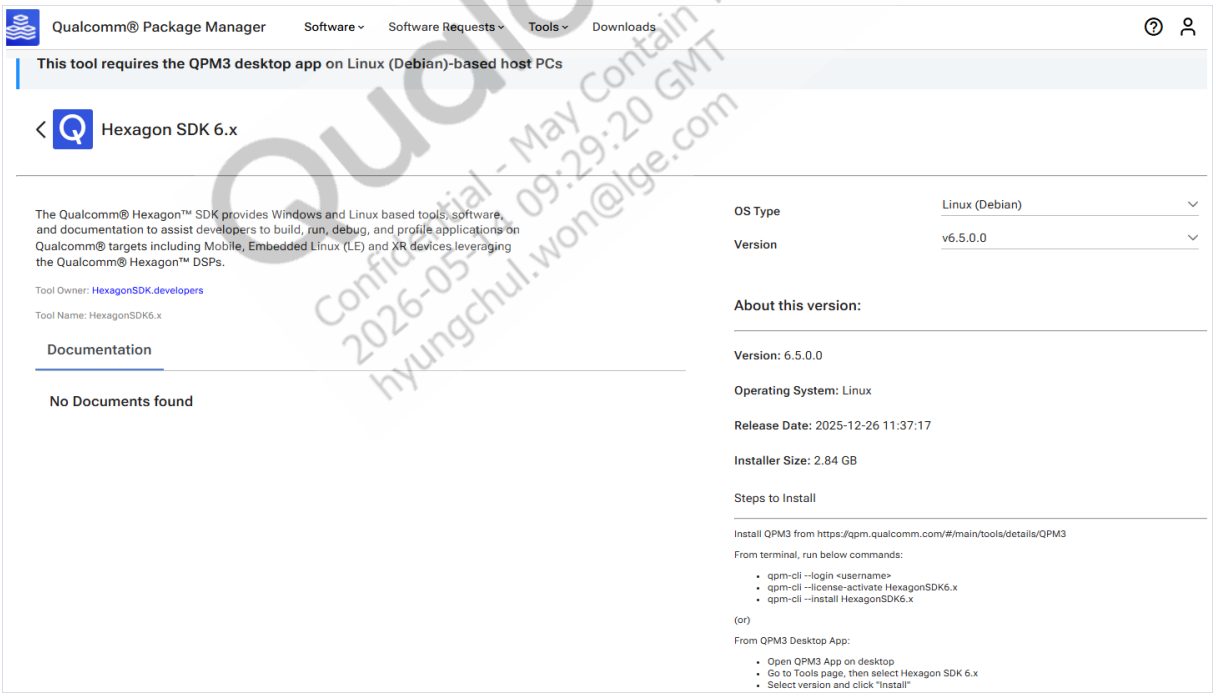
The CPU uses the `clang-14` compiler to build x86_64 targets. You can configure Visual Studio to use a specific version of `clang++` by following the instructions: Clang/LLVM support in Visual Studio projects.

GPU dependencies

The GPU backend kernels use OpenCL. Implement the GPU operations using OpenCL headers. Ensure that you use OpenCL 1.2 or later versions.

HTP dependencies

Compile both HTP and DSP hardware using the Qualcomm® Hexagon™ SDK and Hexagon SDK tools. You can download and install the Hexagon SDK from the Qualcomm Package Manager (QPM).



LPAI dependencies

The LPAI backend supports offline model preparation only.

4 Run AI inference models

This section describes the process to build and convert an AI model for the eNPU hardware accelerator. The AI inference workflow includes multiple stages, such as model preparation and conversion, optimization and quantization, and deployment on the target hardware.

For instructions to download and load the software onto the SW6100/SW6100P target hardware, see Build software.

The following figure shows the model conversion workflow:

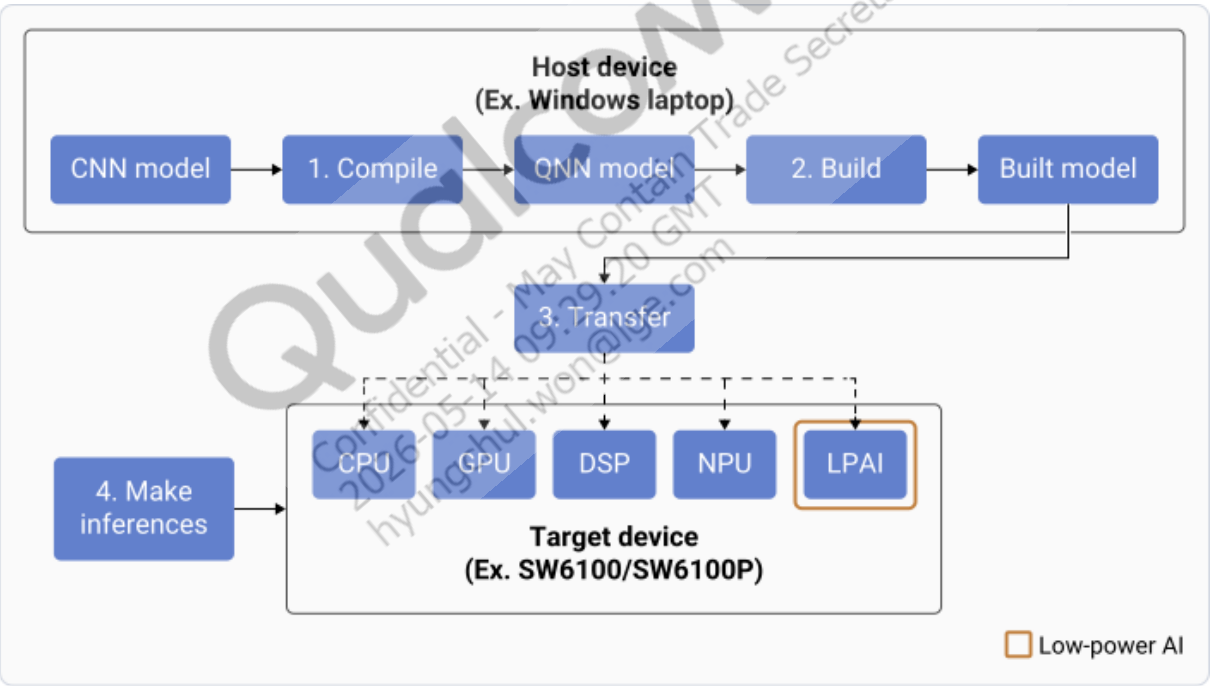


Figure : Model conversion workflow

4.1 Optimize the AI/ML models using eNPU6 design guidelines

The eNPU6 Design Guidelines define design rules and constraints to help AI/ML model designers efficiently offload workloads to the Qualcomm eNPU6 accelerator using the QNN (Qualcomm

Neural Network) framework. These guidelines enable higher eNPU6 utilization, reduced CPU and DSP fallback, optimized performance, memory usage, and power efficiency.

5 Build AI models using QAIRT SDK

This section explains how to convert AI models from frameworks such as ONNX, PyTorch, and TensorFlow into Qualcomm Neural Network (QNN) compatible formats, generate optimized model libraries and context binaries, and prepare models for the low-power AI (LPAI) backend. It explains the Qualcomm AI Runtime (QAIRT) SDK architecture and shows how to build and optimize models for low power AI execution.

The Qualcomm AI Runtime (QAIRT) SDK provides a software architecture for developing and deploying AI/ML use cases on Qualcomm devices across supported AI acceleration cores.

This architecture supports a unified API and modular, extensible per-accelerator libraries, forming a reusable basis for full-stack AI solutions for both Qualcomm and third-party frameworks.

The QAIRT SDK API and the associated software stack provide all the constructs required for an application to build, optimize, and run network models on the preferred hardware accelerator core.

When selecting AI models for QAIRT SDK, ensure that the models align with the supported frameworks and can be efficiently processed through the conversion, optimization, and execution pipeline. As the QAIRT SDK exposes unified, accelerator-specific APIs, selecting models that map correctly to these backends enables smoother graph construction, reliable quantization, and optimal performance on the target hardware.

The following figure shows the components of QAIRT SDK architecture:

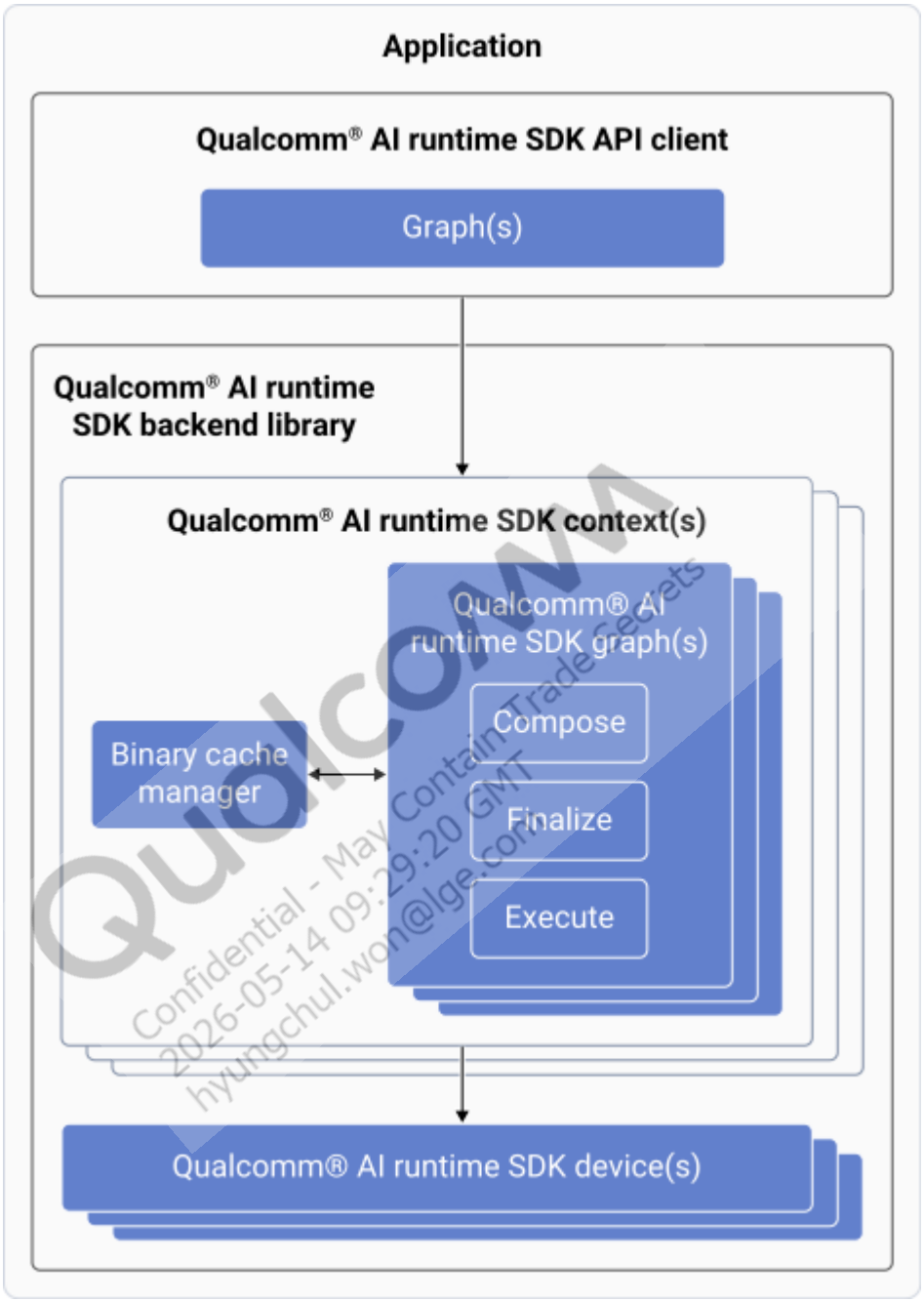


Figure : QAIRT SDK architecture

5.1 Device

A device represents the software abstraction of a hardware accelerator platform. The device provides the constructs required to associate preferred hardware accelerator resources and

execute user-composed graphs. Conceptually, a platform is divided into one or more devices, each of which can include multiple cores.

5.2 Backend

The backend is a top-level API component that hosts and manages most of the backend resources required for graph composition and execution, including an operation registry that stores all available operations.

5.3 Context

A context is a construct that represents all QAIRT SDK components required to sustain a user application. The context hosts user-provided networks, caches constructed entities as serialized objects for reuse, and enables graph interoperability by providing shared memory for tensor exchange.

5.4 Graph

A graph is a direct representation of a loadable network model. The graph consists of nodes that represent operations and tensors that interconnect them to compose a directed acyclic graph. The QAIRT SDK graph construct supports APIs that perform initialization, optimization, and execution of network models.

5.5 QNN workflow

The following figure shows the stages of the Qualcomm Neural Network (QNN) workflow.

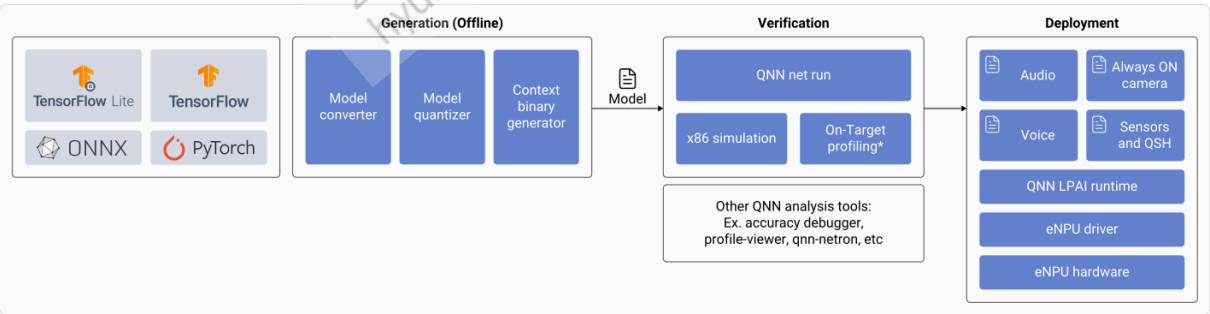


Figure: QNN offline model generation and validation workflow

1. Generate the offline low-power AI (LPAI) model.
2. Validate the LPAI model using `qnn-net-run`.
3. Deploy the model on the target using the QNN APIs.

5.6 Generate LPAI model offline

The QNN workflow to generate the model offline is as follows:

1. Convert the source model into a QNN-compatible format using the QNN Converter tools. For more information, see Convert the models.
   The QNN Converter tool can also perform quantization when provided with a representative input that depicts the behavior of the model. You can override the generated quantization by providing a quantization encoding file.
2. Generate a QNN model library. For more information, see Generate QNN model library.
3. Generate the context binary for the LPAI backend. For more information, see Generate QNN context binary for LPAI backend.
4. Validate the model using the `qnn-net-run` command. For more information, see Validate LPAI model using qnn-net-run.

5.7 Convert the model

You can convert your custom model into a QNN-compatible format (`.cpp` and `.bin`) using the QNN converters.

Qualcomm AI Engine Direct currently supports converters for the following frameworks:

- TensorFlow
- TFLite
- PyTorch
- ONNX

The following figure shows the converter workflow:

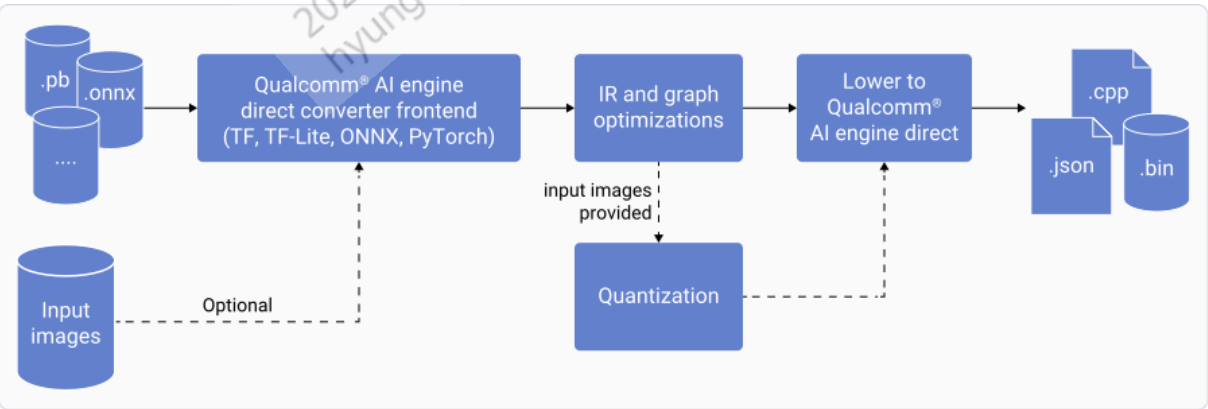


Figure : Workflow of Qualcomm AI Engine Direct converter

Following are the main parts of any converter:

- Front-end translation: Handles converting the original framework model into the common intermediate representation (IR).
- Common IR code: Contains graph and IR operation definitions along with various graph optimizations that translate the graphs.
- Quantizer: Quantizes the model before the final lowering to QNN.
- QNN converter backend: Lowers the IR into the final QNN model API calls.

The converter generates the following output files:

- `.cpp` source file (for example, `model.cpp`): contains the required QNN APIs calls to construct a network graph
- `.bin` binary file (for example, `model.bin`): contains network weights and biases as `float32` data
- `model_net.json`: is a JSON format variant to `model.cpp`

The QNN converter tools provide the following:

- Required arguments for different converters:

```
qnn-onnx-converter --input_network <path_to_source_framework_model.onnx>

qnn-tflite-converter --input_network <path_to_source_framework_model.
tflite>

qnn-tensorflow-converter --input_network < path_to_source_framework_model.
pb> --input_dim <comma_separated_input_dimension_in_NHWC> --out_name
<output_node_name>

qnn-pytorch-converter --input_network < path_to_source_framework_model.pb>
--input_dim <comma_separated_input_dimension_in_NCHW>
```

- Commonly used arguments (supported by all converters):
  - If custom encoding files are required:

    ```
    --quantization_overrides <custom_encodings.json>
    ```

  - To disable Batchnorm folding:

    ```
    --disable_batchnorm_folding
    ```

  - To specify the input data which represents the model behavior (the QNN converter uses this input to determine the float range and the CPU backend processes it for the model quantization):

    ```
    --input_list <path_to_text_file_specifying_input_data>
    ```

  - To preserve the original IO layout of the specified input and output tensors:

```
--preserve_io layout <space_separated_list_of_IO_names>
```

– To disable squashing of ReLU against convolution based operations for quantized models:

```
--disable_relu_squashing
```

– To specify which quantization schema to use for parameters (weight/bias):

```
--param_quantizer_schema <asymmetric/symmetric/unsignedsymmetric>
```

– To specify the quantization schema to use for the activation:

```
--act_quantizer_schema <asymmetric/symmetric/unsignedsymmetric>
```

> **Note:**
>
> `asymmetric` is the default quantization schema for the activation and for the weight/bias.

– Option to select the bit-width to use when quantizing the biases:

```
--bias_bitwidth <8/32>
```

> **Note:**
>
> The default bias bit-width is 8. However, for better accuracy, the recommended default bias bit-width is 32.

– Option to select the bit-width to use when quantizing the activations:

```
--act_bitwidth <8/16>
```

– Option to select the bit-width to use when quantizing the weights:

```
--weights_bitwidth <8/16>
```

– To disable unrolling and expansion of the gated recurrent unit (GRU) operator:

```
--multi_time_steps_gru
```

– To disable unrolling and expansion of the long short-term memory (LSTM) operator:

```
--multi_time_steps_lstm
```

– To specify the target environment as LPAI:

```
--target_backend <HTP/CPU/GPU/HTA/AIC/DSP/**LPAI**>
```

– To enable fallback to floating point instead of fixed point:

```
--float_fallback
```

– To override all input dimensions which leverage variable names:

```
--define_symbol batch_size=<batch_size> seq_len=<seq_len>
```

For more information, see converters.

5.8 Generate QNN model library

The qnn-model-lib-generator tool generates the QNN model library. The tool compiles the QNN model source code into artifacts for a specific target.

The following figure shows the workflow to generate the QNN model library:

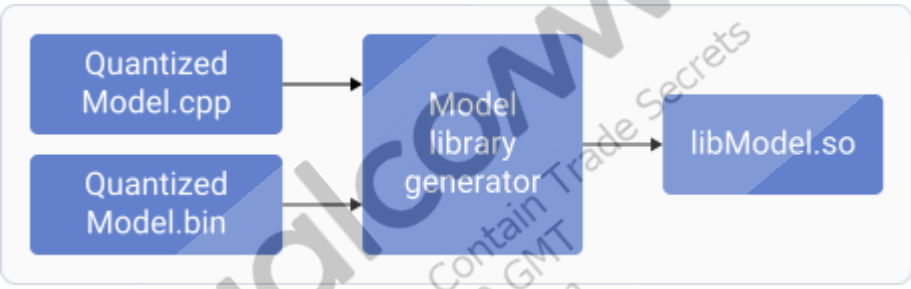


Figure : QNN model generation workflow

Usages:

```
qnn-model-lib-generator [-c <QNN_MODEL>.cpp] [-b <QNN_MODEL>.bin] [-t LIB_
TARGETS] [-l LIB_NAME] [-o OUTPUT_DIR]
```

- Required argument(s):
  - -c <QNN_MODEL>.cpp: File path for the QNN model .cpp file
- Optional argument(s):
  - -b <QNN_MODEL>.bin: File path for the QNN model .bin file
  - -t LIB_TARGETS: Specifies the targets to build the models
  - -l LIB_NAME: Specifies the name to use for libraries
  - -o OUTPUT_DIR: Path for saving output libraries

Select the qnn-model-lib-generator tool from the appropriate x86-64 platform directory ${QNN_SDK_ROOT}/bin/x86_64-<Platform>, where <Platform> corresponds to Windows or Linux.

5.9 Generate QNN context binary for LPAI backend

The qnn-context-binary-generator tool prepares the QNN context binary for the LPAI backend. This tool generates a context binary from a backend-specific model library created by the qnn-model-lib-generator.

```
qnn-context-binary-generator --model <QNN_MODEL>.so

--backend <QNN_BACKEND>.so

--binary_file <BINARY_FILENAME>

--config_file <CONFIG_FILE>.json

--backend_binary <enpu_model>
```

- Required argument(s):
  - --model <QNN_MODEL>.so: Path to QNN model library file
  - --backend <QNN_BACKEND>.so: Path to a QNN backend .so library (libQnnLpai.so)
  - --binary_file <BINARY_FILENAME>: Name of the binary file to save the context binary to with a .bin file extension. If not provided, the tool doesn't create a backend binary.
  - --config_file <CONFIG_FILE>.json: Path to a JSON configuration file related to backend extensions and context priority.
- Optional argument(s):
  --backend_binary <enpu_model>: Name of the binary file to save a backend specific context binary to with .bin file extension. If not provided, the tool doesn't create a backend binary.

> **Note:**
>
> The qnn-context-binary-generator tool must be selected from the appropriate x86 platform directory located at ${QNN_SDK_ROOT}/bin/x86_64-<Platform>, where <Platform> can be either Windows or Linux.

The JSON configuration file is used by the qnn-context-binary-generator tool to obtain details of the LPAI backend extensions.

The following are the contents of the JSON file. The is_persistent_binary flag should be set to TRUE when preparing the model for aDSP.

```
{
 "backend_extensions":
  {
   "shared_library_path":"path_to_libQnnLpaiNetRunExtensions.so",
```

```
   "config_file_path":"path_to_lpai_params.conf"
  },
 "context_configs":
  {
  "is_persistent_binary":true
  }
}
```

Note:

Select the `libQnnLpaiNetRunExtensions.so` from `${QNN_SDK_ROOT}/bin/x86_64-<Platform>`, where `<Platform>` corresponds to Windows or Linux.

The `lpai_params.conf` configuration file contains LPAI backend-specific details. The file has two major fields:

- LPAI backend details used by `qnn-context-binary-generator` during offline preparation.
  - `target_env`: specifies the target environment
  - `enable_hw_ver`: indicates the eNPU hardware version
- LPAI graph details with two graph preparation parameters fields that define:
  - `qnn-context-binary-generator` during offline preparation.
  - `qnn-net-run` during execution.

A sample `lpai_params.conf` file is as follows:

```
{
"lpai_backend": {
   "target_env": "adsp",
   "enable_hw_ver": "v6"
},
"lpai_graph": {
   "execute": {
      "affinity": "hard",
      "core_selection": 2
   }
},
"lpai_private": {
   "disable_async": true
},
"lpai_debug": {
   "eaix_dump": true,
   "enable_ubuf": false,
   "force_nhwc": true
```

```
}
}
```

In this sample `.conf` file:

- All the graph execution parameters use the default values. For more information, see Qualcomm AI Runtime (QAIRT) SDK.
- The `enable_hw_ver` flag specifies the eNPU core version. For SW6100, this version is v6.

The following figure shows the workflow to generate the QNN context binary:

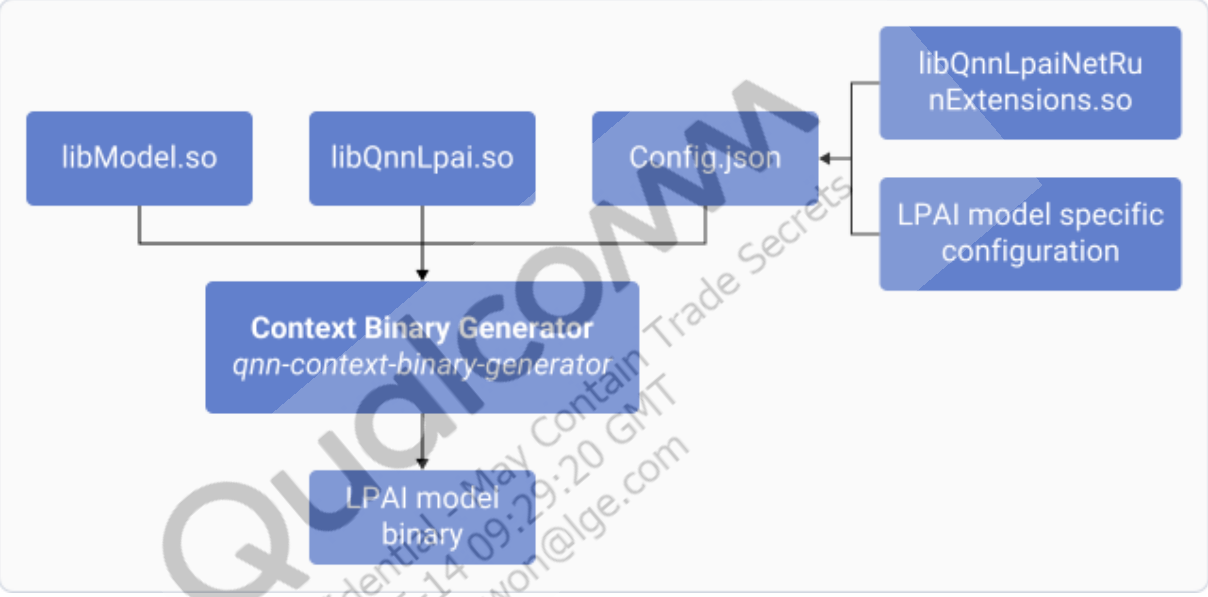


Figure : QNN model context binary workflow

6 Run model inference workflows

This section explains how to select and configure the appropriate execution path, such as direct audio digital signal processor (aDSP)-to-embedded Neural Processing Unit (eNPU), FastRPC, or M55 eNPU mode, to run AI inference efficiently. It also helps you validate models and understand how inference requests flow across processors and subsystems.

You can access the eNPU through multiple execution paths depending on your application architecture. Each path requires specific configuration steps to enable efficient offload. These paths determine how inference requests route among the application processor, aDSP, and the eNPU hardware.

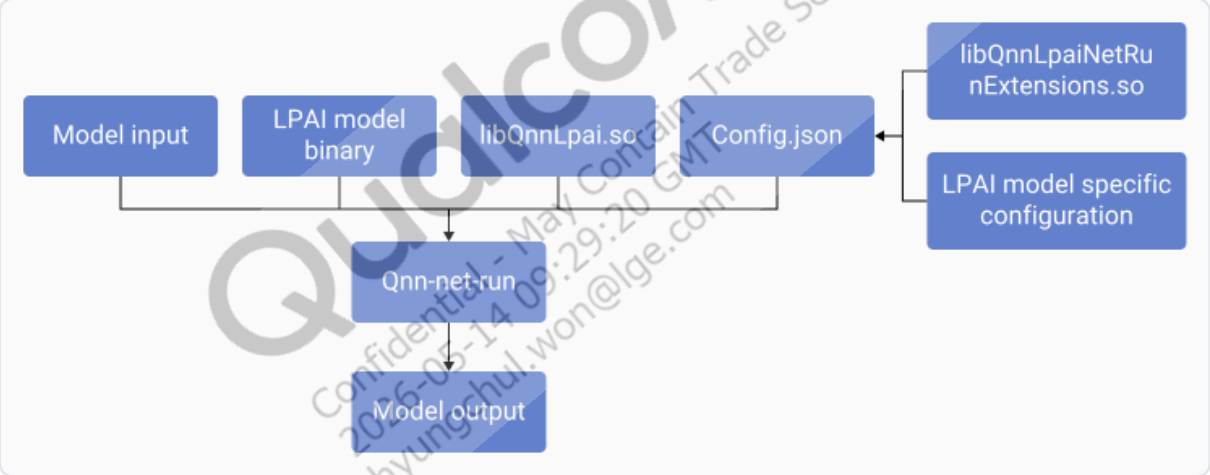


Figure: Workflow of QNN-LPAI model executions

For more information, see qnn-net-run.

6.1 Prerequisites

To access the eNPU through aDSP, CPU, and M55 hardware, ensure that you complete the following steps:

1. Run the `setup_sdk_env` script from the Hexagon SDK root directory located on the host machine:

```
HEXAGON_SDK_ROOT="/local/mnt/workspace/Qualcomm/Hexagon_SDK/
6.5.0.0"source "${HEXAGON_SDK_ROOT}/setup_sdk_env.source"
```

1. Ensure that you sign the `lpai-v6` and `hexagon-v81` aDSP libraries before pushing them to the device:

```
python <Hexagon_SDK>/utils/scripts/signer.py sign -i <lib_name.so>
-o <path_to_signed_lib>
```

   - Repeat the command for each library that requires signing for the selected execution mode.

6.2 Direct aDSP-to-eNPU mode

In the direct aDSP-to-eNPU mode, the application runs directly on the aDSP or loads as an aDSP service. The application links the Qualcomm Neural Network (QNN) low-power AI (LPAI) backend natively on aDSP and communicates directly with the eNPU driver, eliminating CPU-side inter-processor communication (IPC) during inference.

The key characteristics of this mode are:

- aDSP acts as both client and executor
- QNN backend communicates directly with the eNPU
- FastRPC doesn't enter the steady state

The list of libraries required to enable direct aDSP-to-eNPU mode are as follows:

Linux Android Wear

- `libQnnLpaiIsland.so (lpai-v6)`
- `libQnnLpaiNetRunExtensions.so (lpai-v6)`
- `libQnnLpai.so (lpai-v6)`
- `libQnnNetRunDirectV81Skel.so (hexagon-v81)`
- `libQnnHexagonSkel_dspApp.so (hexagon-v81)`
- `libQnnSystem.so (hexagon-v81)`
- `libQnnNetRunDirectV81Stub.so (aarch64-android)`

Linux Embedded

- `libQnnLpai.so (lpai-v6)`
- `libQnnLpaiIsland.so (lpai-v6)`
- `libQnnLpaiNetRunExtensions.so (lpai-v6)`
- `libQnnNetRunDirectV81Skel.so (hexagon-v81)`
- `libQnnNetRunDirectV81Stub.so (aarch64-oe-linux-gcc11.2)`
- `libQnnSystem.so (hexagon-v81)`

- `libQnnHexagonSkel_dspApp.so (hexagon-v81)`

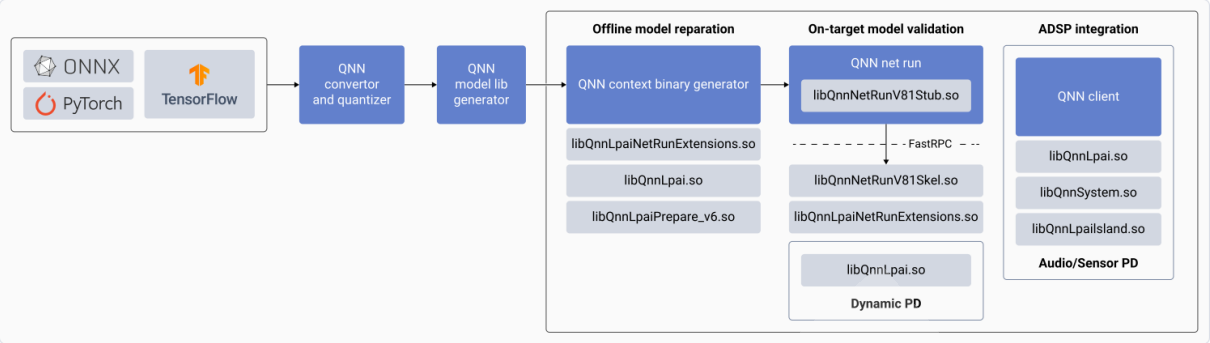


Figure : QNN model preparation, validation, and integration flow

To run the direct aDSP-to-eNPU mode, do the following:

1. Set up the host environment for the target architecture and hardware version using the following commands:

Linux Android Wear

```
export QNN_TARGET_ARCH=aarch64-android

export HW_VER=v6

export DSP_ARCH=hexagon-v81

export DSP_VER=V81
```

Linux Embedded

```
export QNN_TARGET_ARCH=aarch64-oe-linux-gcc11.2

export HW_VER=v6

export DSP_ARCH=hexagon-v81

export DSP_VER=V81
```

2. Prepare the `qnn_param_direct.json` and `lpai_param_direct.conf` files as follows:

- `qnn_param_direct.json`

```
{
    "backend_extensions": {
```

```
    "shared_library_path": "./libQnnLpaiNetRunExtensions.so",
    "config_file_path": "./lpai_param_direct.conf"
    },
    "context_configs": {"is_persistent_binary": true }
}
```

- `lpai_param_direct.conf`

```
{
    "lpai_backend": {
        "target_env": "adsp",
        "enable_hw_ver": "v6"
    },
    "lpai_graph": {
        "prepare": {}
    },
    "lpai_debug": {
        "eaix_dump": true
    },
    "lpai_private": {"disable_async": true}
}
```

3. Create a directory using the following commands:

Linux Android Wear

```
adb shell mkdir -p  /data/qnn/qnn_law_direct
adb shell mkdir -p  /data/qnn/models
```

Linux Embedded

```
adb shell mkdir -p  /data/qnn/qnn_le_direct
adb shell mkdir -p  /data/qnn/models
```

4. Push the following preferred libraries from the QNN SDK to the target device:

Linux Android Wear

```
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/
libQnnNetRunDirectV81Stub.so /data/qnn/qnn_law_direct
adb push ${QNN_SDK_ROOT}/lib/lpai-${HW_VER}/unsigned/
libQnnLpaiIsland.so /data/qnn/qnn_law_direct
adb push ${QNN_SDK_ROOT}/lib/lpai-${HW_VER}/unsigned/libQnnLpai.so
/data/qnn/qnn_law_direct
```

```
adb push ${QNN_SDK_ROOT}/lib/lpai-${HW_VER}/unsigned/
libQnnLpaiNetRunExtensions.so /data/qnn/qnn_law_direct
adb push ${QNN_SDK_ROOT}/lib/${DSP_ARCH}/unsigned/
libQnnHexagonSkel_dspApp.so  /data/qnn/qnn_law_direct
adb push ${QNN_SDK_ROOT}/lib/${DSP_ARCH}/unsigned/
libQnnNetRunDirectV81Skel.so  /data/qnn/qnn_law_direct
adb push ${QNN_SDK_ROOT}/lib/${DSP_ARCH}/unsigned/libQnnSystem.so
/data/qnn/qnn_law_direct
adb push ${QNN_SDK_ROOT}/bin/${QNN_TARGET_ARCH}/qnn-net-run  /
data/qnn/models/
```

Linux Embedded

```
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/
libQnnNetRunDirectV81Stub.so /data/qnn/qnn_le_direct
adb push ${QNN_SDK_ROOT}/lib/lpai-${HW_VER}/unsigned/
libQnnLpaiIsland.so /data/qnn/qnn_le_direct
adb push ${QNN_SDK_ROOT}/lib/lpai-${HW_VER}/unsigned/libQnnLpai.so
/data/qnn/qnn_le_direct
adb push ${QNN_SDK_ROOT}/lib/lpai-${HW_VER}/unsigned/
libQnnLpaiNetRunExtensions.so /data/qnn/qnn_le_direct
adb push ${QNN_SDK_ROOT}/lib/${DSP_ARCH}/unsigned/
libQnnHexagonSkel_dspApp.so  /data/qnn/qnn_le_direct
adb push ${QNN_SDK_ROOT}/lib/${DSP_ARCH}/unsigned/
libQnnNetRunDirectV81Skel.so  /data/qnn/qnn_le_direct
adb push ${QNN_SDK_ROOT}/lib/${DSP_ARCH}/unsigned/libQnnSystem.so
/data/qnn/qnn_le_direct
adb push ${QNN_SDK_ROOT}/bin/{QNN_TARGET_ARCH}/qnn-net-run  /data/
qnn/models/
```

5. Push the configuration files to the target device:

```
adb push qnn_param_direct.json /data/qnn/models/
adb push lpai_param_direct.conf /data/qnn/models
```

6. Push the quantized serialized context binary to the device, including the input list and raw input:

```
adb push .\<models from host >  /data/qnn/models/
adb push .\<input.txt >  /data/qnn/models/
adb push .\<input.raw >  /data/qnn/models/
adb shell "chmod 777 /data/qnn/models/qnn-net-run"
```

7. Run the LPAI model using `qnn-net-run`:

Linux Android Wear

```
adb shell
export LD_LIBRARY_PATH=/data/qnn/qnn_law_direct
export ADSP_LIBRARY_PATH=/data/qnn/qnn_law_direct
export LD_LIBRARY_PATH=/vendor/lib64:$LD_LIBRARY_PATH
cd /data/qnn/models/
./qnn-net-run --direct_mode adsp --input_list input.txt --
profiling_level basic --backend libQnnLpai.so --config_file qnn_
params_direct.json  --retrieve_context <model>.bin
```

Linux Embedded

```
adb shell
export LD_LIBRARY_PATH=/data/qnn/qnn_le_direct
export ADSP_LIBRARY_PATH=/data/qnn/qnn_le_direct
cd /data/qnn/models/
./qnn-net-run --direct_mode adsp --input_list input.txt --
profiling_level basic --backend libQnnLpai.so --config_file qnn_
params_direct.json  --retrieve_context <model>.bin
```

8. Pull the output file to check the results:

```
adb pull /data/qnn/models/output .\<HOST PATH>
```

6.3 eNPU FastRPC mode

In the eNPU FastRPC mode, the application runs on the CPU. FastRPC forwards all QNN API calls to the aDSP. The aDSP receives these calls and invokes the QNN LPAI backend, which finally communicates to the eNPU driver for hardware execution.

The key characteristics of this mode are:

- CPU is the client
- Uses FastRPC `stub` or `skel` files to cross the aDSP
- aDSP acts as a proxy to the eNPU

The list of libraries required to enable eNPU FastRPC mode is as follows:

Linux Android Wear

- `libQnnLpaiSkel.so (lpai-v6)`
- `libQnnLpai.so (aarch64-android)`
- `libQnnLpaiNetRunExtensions.so (aarch64-android)`
- `libQnnLpaiStub.so (aarch64-android)`
- `libQnnSystem.so (aarch64-android)`

Linux Embedded

- `libQnnLpai.so (aarch64-oe-linux-gcc11.2)`
- `libQnnLpaiNetRunExtensions.so (aarch64-oe-linux-gcc11.2)`
- `libQnnLpaiSkel.so (lpai-v6)`
- `libQnnLpaiStub.so (aarch64-oe-linux-gcc11.2)`
- `libQnnSystem.so (aarch64-oe-linux-gcc11.2)`

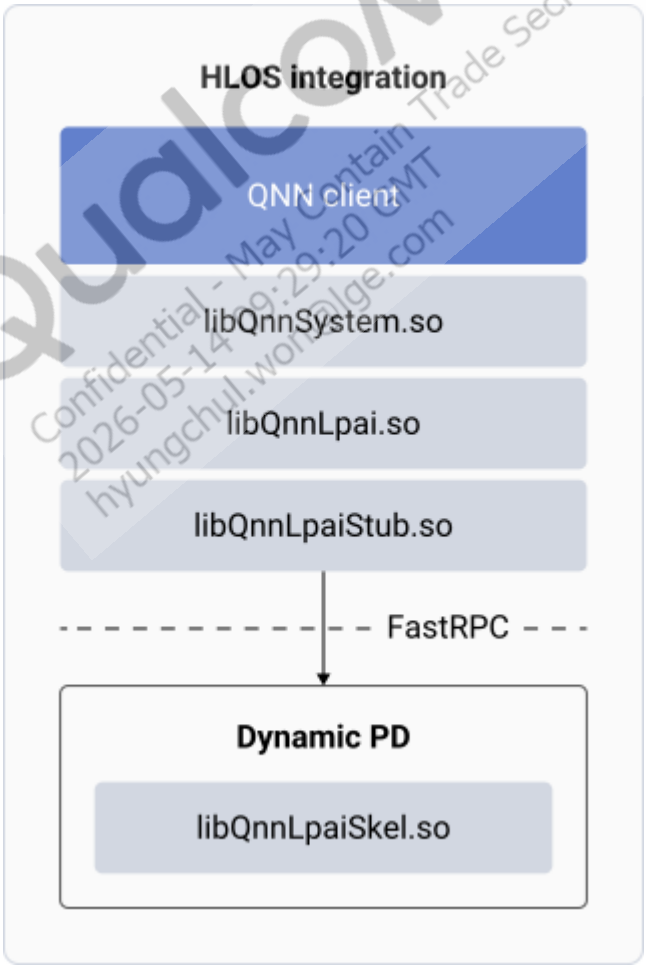


Figure : QNN client integration in high-level operating system

To run the eNPU FastRPC mode, do the following:

1. Set up the host environment using the following commands:

   Linux Android Wear

   ```
   export QNN_TARGET_ARCH=aarch64-android
   export HW_VER=v6
   ```

   Linux Embedded

   ```
   export QNN_TARGET_ARCH=aarch64-oe-linux-gcc11.2
   export HW_VER=v6
   ```

2. Prepare the configuration files as follows:

   - `qnn_config.json`

     ```
     {
        "backend_extensions": {
           "shared_library_path": "<PATH>/
     libQnnLpaiNetRunExtensions.so",
           "config_file_path": "<PATH>/lpai_config.conf"
        }
     }
     ```

   - `lpai_config.conf`

     ```
     {
        "lpai_backend": {
            "target_env": "adsp",
            "enable_hw_ver": "v6"
        },
        "lpai_graph": {
            "prepare": {}
        },
        "lpai_debug": {
            "eaix_dump": true
        }
     }
     ```

3. Create a directory using the following commands:

Linux Android Wear

```
adb shell mkdir -p  /data/qnn/qnn_law_fastrpc
adb shell mkdir -p  /data/qnn/models
```

Linux Embedded

```
adb shell mkdir -p  /data/qnn/qnn_le_fastrpc
adb shell mkdir -p  /data/qnn/models
```

4. Push the required libraries from the QNN SDK to the target device:

Linux Android Wear

```
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/libQnnLpai.so /data/qnn/
qnn_law_fastrpc/
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/libQnnLpaiNetRunExtensions.
so /data/qnn/qnn_law_fastrpc/
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/libQnnLpaiStub.so /data/
qnn/qnn_law_fastrpc/
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/libQnnSystem.so /data/qnn/
qnn_law_fastrpc/
adb push ${QNN_SDK_ROOT}/lib/lpai-${HW_VER}/unsigned/libQnnLpaiSkel.so /
data/qnn/qnn_law_fastrpc
adb push ${QNN_SDK_ROOT}/bin/{QNN_TARGET_ARCH}/qnn-net-run  /data/qnn/
models/
```

Linux Embedded

```
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/libQnnLpai.so /data/qnn/
qnn_le_fastrpc/
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/libQnnLpaiNetRunExtensions.
so /data/qnn/qnn_le_fastrpc/
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/libQnnLpaiStub.so /data/
qnn/qnn_le_fastrpc/
adb push ${QNN_SDK_ROOT}/lib/${QNN_TARGET_ARCH}/libQnnSystem.so /data/qnn/
qnn_le_fastrpc/
adb push ${QNN_SDK_ROOT}/lib/lpai-${HW_VER}/unsigned/libQnnLpaiSkel.so /
data/qnn/qnn_le_fastrpc
adb push ${QNN_SDK_ROOT}/bin/{QNN_TARGET_ARCH}/qnn-net-run  /data/qnn/
models/
```

5. Push the configuration files to the target device:

```
adb push qnn_config.json /data/qnn/models/
adb push lpai_config.conf /data/qnn/models/
```

6. Push the quantized serialized context binary to the device including the input list and raw input:

```
adb push .\<models from host >  /data/qnn/models/
adb push .\<input.txt >  /data/qnn/models/
adb push .\<input.raw >  /data/qnn/models/
adb shell "chmod 777 /data/qnn/models/qnn-net-run"
```

7. Run the LPAI model using qnn-net-run:

Linux Android Wear

```
adb shell
   export LD_LIBRARY_PATH=/data/qnn/qnn_law_fastrpc
   export ADSP_LIBRARY_PATH=/data/qnn/qnn_law_fastrpc
   export LD_LIBRARY_PATH=/vendor/lib64:$LD_LIBRARY_PATH
   cd /data/qnn/models/
   ./qnn-net-run --input_list input.txt --profiling_level basic --backend
libQnnLpai.so --config_file qnn_config.json  --retrieve_context <model>.bin
```

Linux Embedded

```
adb shell
   export LD_LIBRARY_PATH=/data/qnn/qnn_le_fastrpc
   export ADSP_LIBRARY_PATH=/data/qnn/qnn_le_fastrpc
   cd /data/qnn/models/
   ./qnn-net-run --input_list input.txt --profiling_level basic --backend
libQnnLpai.so --config_file qnn_config.json  --retrieve_context <model>.bin
```

8. Pull the output file to check the results:

```
adb pull /data/qnn/models/output .\<HOST PATH>
```

6.4 M55 eNPU mode

Use this execution path to offload ML inference from the M55 processor to the eNPU. The application runs a QNN client on M55 and issues inference requests using QNN APIs. The QNN proxy serializes the QNN API calls and parameters and transfers the requests to the Q6 processor using IPC over GLINK. On the Q6 processor, the QNN LPAI libraries invoke the eNPU driver and manage execution of the inference workload on the eNPU. After inference completes, the proxy serializes the results and returns them to the M55 client, which receives and processes the output.

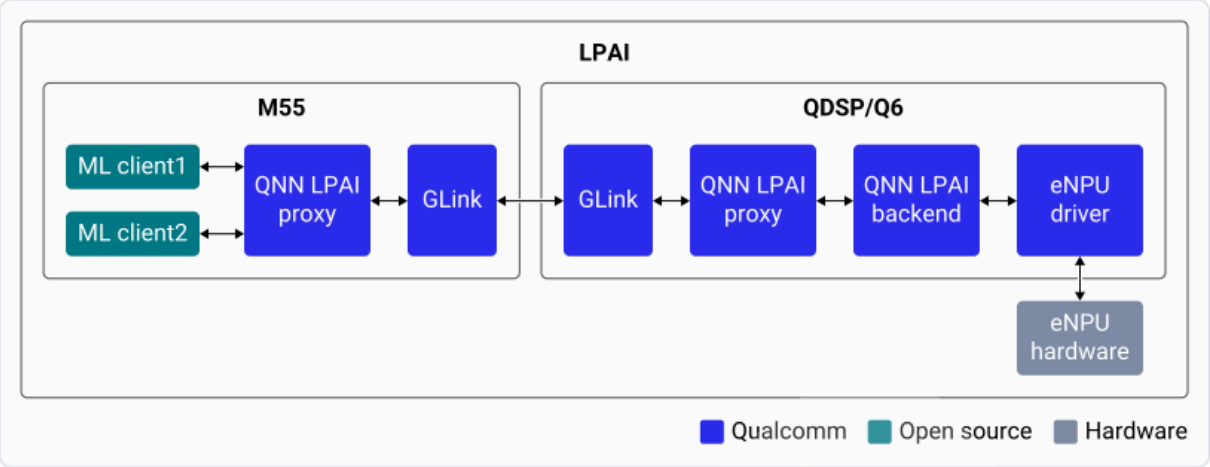


Figure : QNN proxy workflow for M55 eNPU mode

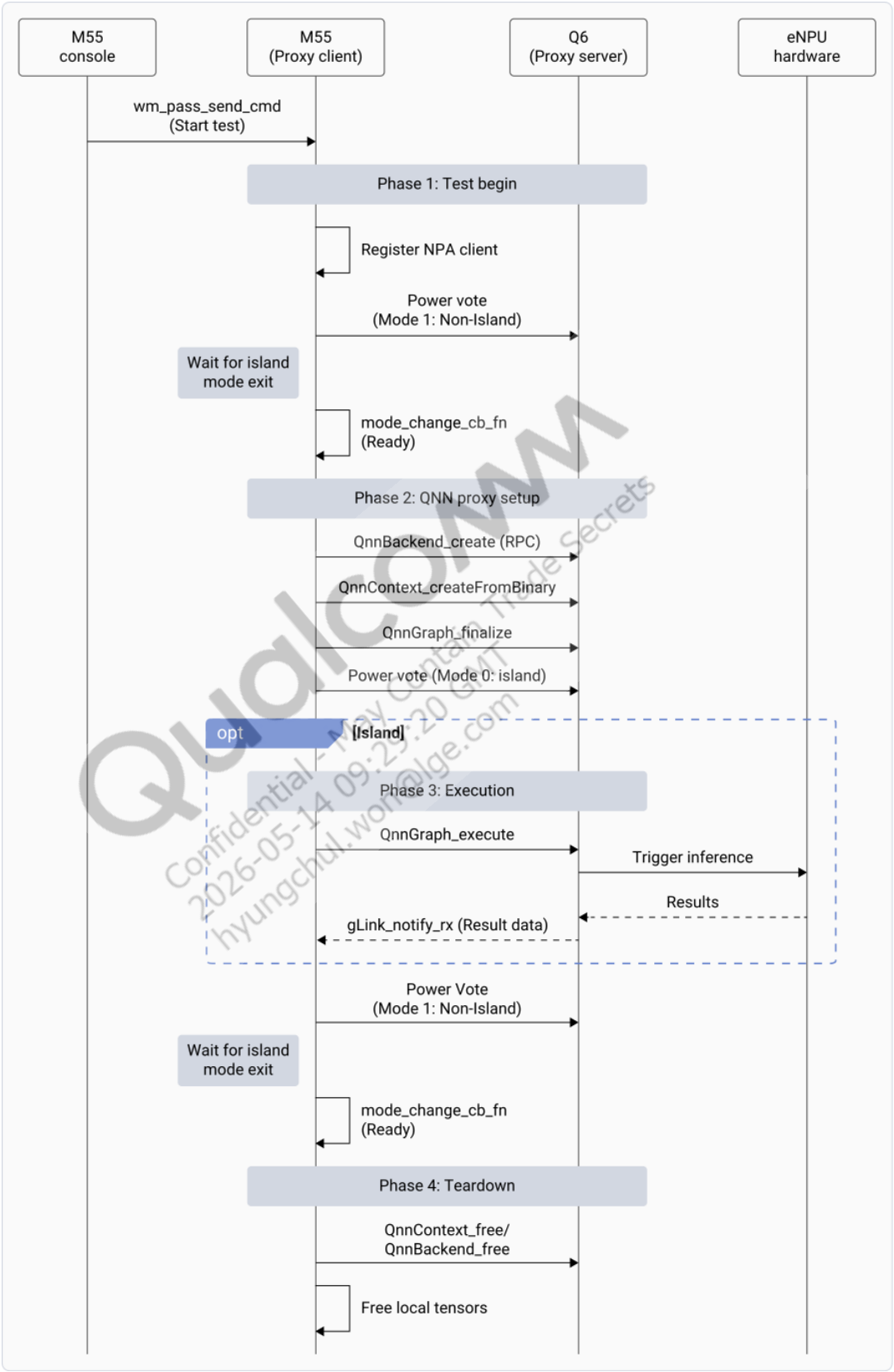


Figure : M55eNPU offload call flow

> **Note:**
>
> Linux Android Wear platform doesn't support the M55 eNPU mode.

To run the M55 eNPU mode, do the following:

1. Disable the SELinux policy (SEPolicy) and enable the M55 serial port:

Linux Android Wear

```
adb shell setenforce 0
adb root
adb shell lpi_utility_test 1 1
```

Linux Embedded

```
adb shell setenforce 0
adb shell aon_utility_cli 1 1
```

2. Set up the host environment using the following commands:

```
export DSP_ARCH=hexagon-v81
```

3. Prepare the target device by verifying required libraries:

   Linux Android Wear

   Verify that the following required libraries are available on the device. Push these libraries from the SDK only if they are not already present in the target device.

   - `./vendor/dsp/adsp/libQnnLpai.so`
   - `./vendor/dsp/adsp/libQnnSystem.so`
   - `./vendor/dsp/adsp/libQnnLpaiSkel.so`
   - `./vendor/dsp/adsp/libQnnLpaiIsland.so`

Linux Embedded

Push the configuration files, data files, and the `libQnnSystem.so (hexagon-v81)` library from the QNN SDK to the target device:

```
adb push ${QNN_SDK_ROOT}/lib/${DSP_ARCH}/unsigned/libQnnSystem.so  /usr/
lib/rfsa/adsp
```

> **Note:**
>
> Ensure to sign in the libraries before pushing them to the target device.

4. Create the `config.txt` input configuration file and push it to the device. This file defines the use case parameters, including the model, input and output data, and execution settings. The following table lists the options supported by the configuration file:

Table : Configuration file parameters

Parameters	Definition
`model_file`	Specifies the path to the model file
`input_file`	Defines the path to the input raw data files
`golden_file`	Sets the path to the output raw data files
`fps` (Optional)	Sets the preferred frames per second (fps) for model execution
`ftrt_ratio` (Optional)	Specifies the eNPU frequency. The first 4 bits are for frequency, and the last 12 are for the `ftrt_ratio`. The supported values are: • `0x1000`: For low static voltage scaling (SVS) frequency mode • `0x8000`: For turbo frequency mode
`affinity` (Optional)	Sets the core affinity mode in the range <1-2>: • 1: For SOFT mode, allows migration. • 2: For HARD mode, pins to the eNPU cores.
`core_selection` (Optional)	Sets the eNPU core preference in the range <0-2>: • 0: For any eNPU • 1: For Little eNPU • 2: For Big eNPU
`exec_count` (Optional)	Specifies the total number of inferences required for the execution. The value is an integer.
`client_type` (Optional)	Specifies the client type in the range <1-2>: • 1: For `REAL_TIME` • 2: For `NON_REAL_TIME`
`mem_type` (Optional)	Specifies the memory type for model, input, and output tensors in the range <0-1>: • 0: For `LPASS_TCM` • 1: For `PSRAM`

An example `config.txt` file is as follows:

```
model_file /<Model path>/<Model name>.bin -input_file /<raw input
path>/input0.raw -num_inputs 1 -fps 30 -ftrt_ratio 0x1000 -
affinity 1 -core_selection 0 -num_inferences 10 -client_type 1
```

5. Push the data to the target device using the following `config.txt`:

```
adb shell mkdir /etc/sensors/sdk/
adb push <compiled model from host .bin> /etc/sensors/sdk/
adb push <raw input file from host input0.raw > /etc/sensors/sdk/
adb push <cfg_file file from host le_config.txt > /etc/sensors/sdk/
```

6. To test the M55-to-eNPU execution, connect the serial cable to the device and identify the M55 UART COM port in the Device Manager. Open the corresponding COM port with a baud rate of 4000000, and run the following command on the M55 UART `config.txt`:

```
wm_pass send_cmd "qnn_proxy_test -cfg_file /etc/sensors/sdk/le_config.txt"
```

   The `wm_pass` command uses the Wearable Manager to instruct the M55 sensor hub to run `qnn_proxy_test`. The application reads the configuration file and sends ML inference requests to the aDSP, where the QNN LPAI backend runs the model on the eNPU.

6.5 Validate LPAI model using qnn-net-run on host computer

You can validate a QNN-LPAI model with `qnn-net-run` using the following methods:

- LPAI x86 Linux/Windows simulation: This mode supports both offline model preparation and direct execution using `qnn-net-run`.
  - Direct execution – In this mode, `qnn-net-run` internally prepares a graph and executes it on x86.

```
${QNN_SDK_ROOT}/bin/x86_64-<Platform>/qnn-net-run

--backend libQnnLpai.so

--input_list input_list.txt

--model <libQnnModel>.so

--config_file <qnn_config.json>
```

> **Note:**
>
> Build the `<libQnnModel.so>` for the x86 platform (Windows or Linux). Based on the platform, select the appropriate `qnn-net-run` from the corresponding folders (`x86_64-linux-clang` or `x86_64-windows-msvc`). For more information, see Generate QNN model library.

  - Using serialized context binary. Usages:

```
${QNN_SDK_ROOT}/bin/x86_64-<platform>/qnn-net-run

--backend libQnnLpai.so

--input_list input_list.txt

--retrieve_context  <lpai_model>.bin

--config_file <qnn_config.json>
```

Note:

Build the `<lpai_model>.bin` for the x86 platform (Windows or Linux). Based on the platform, select the appropriate `qnn-net-run` from the corresponding folders (`x86_64-linux-clang` or `x86_64-windows-msvc`). For more information, see Generate QNN context binary for LPAI Backend.

7 Run MobileNetV2 model workflow

This section provides an end-to-end example that demonstrates how to prepare, convert, run, and postprocess a `MobileNetV2` model on the low-power AI (LPAI) backend using the embedded Neural Processing Unit (eNPU).

The preprocessing step ensures that the eNPU receives clean, quantized, correctly shaped, and memory-aligned input tensors. You must transform any real-world input such as images, audio, or sensor data into this strict format so that the QAIRT SDK can run efficiently on the eNPU.

The following figure represents the workflow to generate `MobileNet v2 onnx` model from `torchvision` and quantize it to run on low-power AI (LPAI):

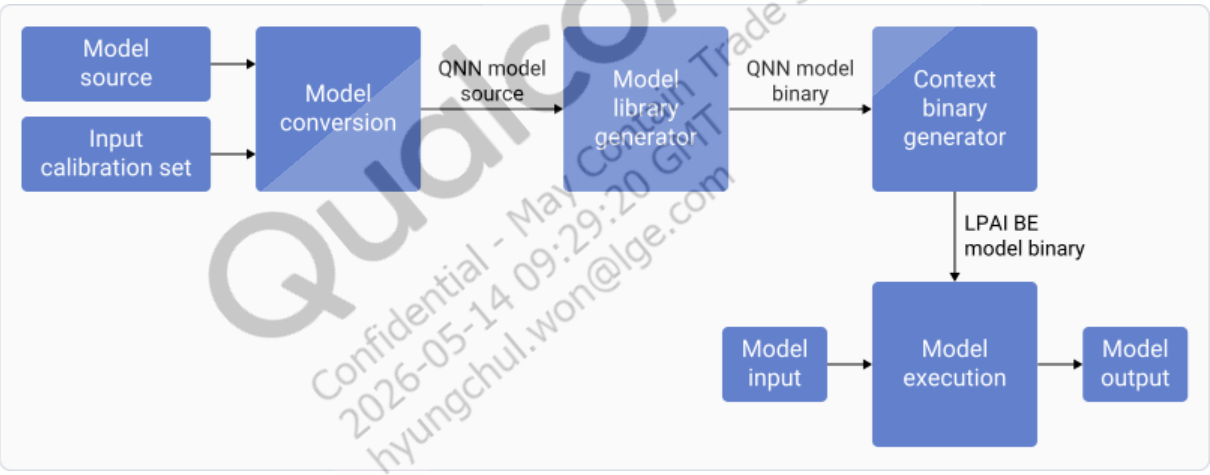


Figure: QNN-LPAI high-level workflow

7.1 Prepare the MobileNetV2 models and input

Prepare the model so that its structure, inputs, and parameters align with the requirements of the LPAI workflow. This preparatory step allows the toolchain to reliably process the model.

To prepare the models, run the following commands on the target device. Ensure that you use `onnx_model` as a fixed string literal and not as a variable name.

```
import torch
import torchvision.models as models
model = models.mobilenet_v2(pretrained=True)
```

```
model.eval()
dummy_input = torch.randn(1, 3, 224, 224)
onnx_model = "model.onnx"
# Export to ONNX
torch.onnx.export(
model,
dummy_input,
onnx_model,
export_params=True,
opset_version=11,
do_constant_folding=True,
input_names=['input'],
output_names=['output']
)
print(f"model exported to {onnx_model}!")
# Create sample input float32 raw file dummy_input.numpy().tofile("sample_
input.raw")
# Create input list file
with open(f'./input_list.txt', 'w') as f: f.write(f'input:=sample_input.raw\n')
```

7.2 Convert the MobileNetV2 model

Convert the prepared model into the format expected by the LPAI toolchain, ensuring compatibility with the compiler and runtime components.

1. Prepare the required `.conf` file:

```
{
    "lpai_backend": {
    "target_env": "adsp",
    "enable_hw_ver": "v6"
    },
    "lpai_graph": {
    "execute": {
        "affinity": "hard",
        "core_selection": 2
    }
    },
    "lpai_private": {
    "disable_async": true },
    "lpai_debug": {
    "eaix_dump": true,
    "enable_ubuf": false,
    "force_nhwc": true
```

```
    }
    }
```

2. To convert the models, run the following commands on your host computer:

```
source ${QNN_SDK_ROOT}/bin/envsetup.sh
```

```
qnn-onnx-converter --input_network model.onnx --act_bitwidth 8 --
weights_bitwidth 8 --bias_bitwidth 32 --target_backend LPAI --
input_ list input_list.txt --act_quantizer_schema symmetric --
param_ quantizer_schema symmetric --disable_relu_squashing --
disable_ batchnorm_folding
```

```
qnn-model-lib-generator -b model.bin -c model.cpp -t x86_64-linux-
clang
```

```
qnn-context-binary-generator --model x86_64-linux-clang/libmodel.
so --backend ${QNN_SDK_ROOT}/lib/x86_64-linux-clang/libQnnLpai.so
--binary_file model_lpai --config_file qnn_config.json --output_
dir
```

For more details about `qnn_config.json`, see Generate QNN context binary for LPAI backend.

7.3 Run and profile MobileNetV2 model

Run and profile the model to confirm functional correctness and collect performance data that informs subsequent preprocessing and optimization work.

For more information about target device setup, see Validate LPAI model using qnn-net-run.

- To run and profile AI models, use the following command:

```
./qnn-net-run --backend libQnnLpai.so --retrieve_context
./model.bin
   --device_options device_id:0  --input_list input_list.txt
```

7.4 Postprocess the MobileNetV2 output

After you run the model on the device, the inference tool generates raw output tensor files and stores them in the designated output directory. Once the inference is complete, you must transfer these results from the device to your host computer and perform the necessary postprocessing steps, such as sorting the output scores to identify the top predictions.

- To postprocess the output, run the following commands on the target device:

```
adb pull /data/local/tmp/output
```

```
import numpy as np
output = np.fromfile("output/Result_0/output.raw", np.float32)
sorted_score = np.argsort(output)[::-1]
print(sorted_score[:5])
```

For information about deploying and evaluating ML models on the LPAI backend, see LPAINotebookExamples.

8 Run sample application

This section explains how to build and run the SampleAppLPAI reference application to evaluate the Qualcomm Neural Network (QNN) low-power AI (LPAI) backend on Linux and Hexagon DSP platforms. It covers environment setup, compilation, and execution procedures to help you validate your development setup across different targets.

8.1 Prerequisites

Ensure that your host computer has the following:

- QAIRT SDK installed, with the `QNN_SDK_ROOT` environment variable set.
- Hexagon SDK installed, with the `HEXAGON_SDK_ROOT` environment variable set.
- GCC toolchain (required for Yocto-based Linux targets).
- Clang compiler (required for x86 Linux builds).

8.2 Locate the reference sample application

The reference implementation is located in the following directory:

```
${QNN_SDK_ROOT}/examples/QNN/SampleApp/SampleAppLPAI
```

This directory contains source code and build scripts for the sample application. The directory serves as a practical example of how to use the LPAI backend with the QNN SDK.

8.3 Build and compile the sample application

The following section describes the procedure to build and compile the `SampleAppLPAI` reference application on the Linux and Hexagon DSP targets.

8.3.1 Linux setup

You can build `qnn-sample-app` on Linux using the Clang compiler

1. If the Clang compiler isn't available in your system *PATH*, use the SDK-provided script to install it:

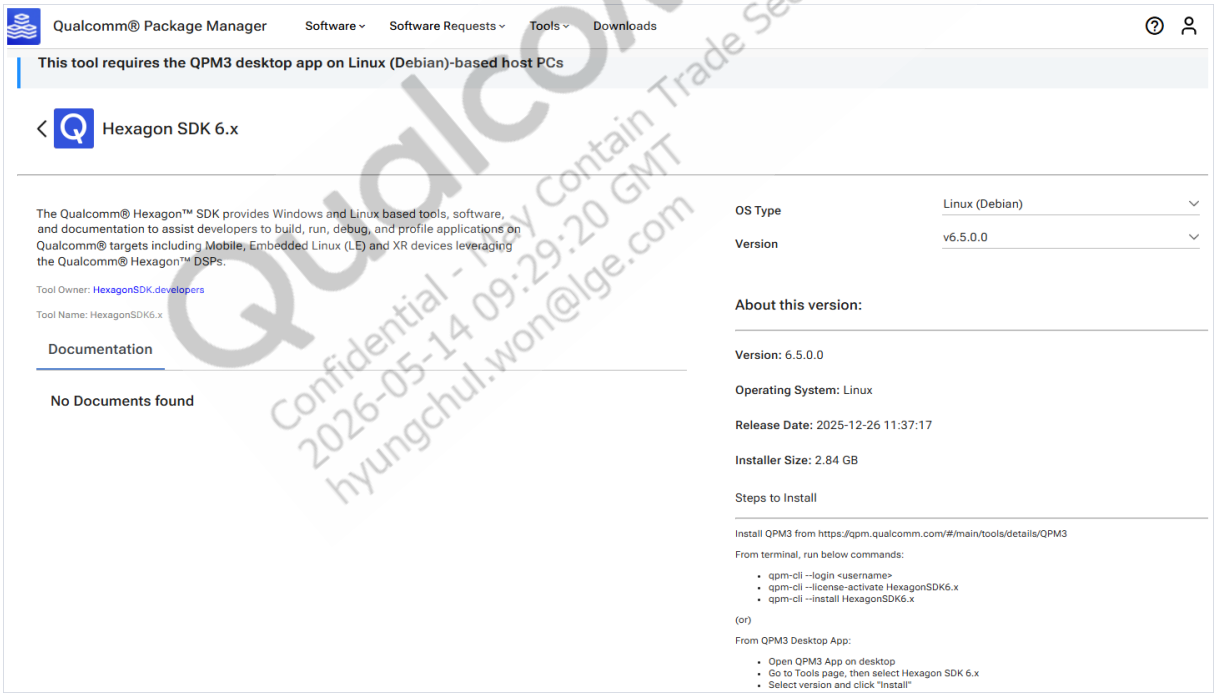
```
$ sudo bash ${QNN_SDK_ROOT}/bin/check-linux-dependency.sh
```

2. To verify the environment setup, run the following command:

```
$ ${QNN_SDK_ROOT}/bin/envcheck -n
```

8.3.2 Hexagon setup

1. To build for Hexagon DSP, install the Hexagon SDK from the Qualcomm package manager (QPM) and configure it.



1. To set up the environment, run the following command:

```
$ source ${HEXAGON_SDK_ROOT}/setup_sdk_env.source
```

For more information, see LPAI.

8.3.3 Compile the build

- To compile and build the sample LPAI application for x86-64 Linux, run the following commands from the SampleAppLPAI directory:

```
cd ${QNN_SDK_ROOT}/examples/QNN/SampleApp/SampleAppLPAI
make all_x86
```

8.4 Run the sample application on Linux host

- To run the sample application on Linux host computer, update the library path and run the following commands from the SampleAppLPAI directory:

```
export LD_LIBRARY_PATH=${QNN_SDK_ROOT}/lib/x86_64-linux-clang
```

```
${QNN_SDK_ROOT}/examples/QNN/SampleApp/SampleAppLPAI/bin/x86_64-linux-clang/
qnn-sample-app \
--backend ${QNN_SDK_ROOT}/lib/x86_64-linux-clang/libQnnLpai.so \
--retrieve_context <model>.bin \
--systemlib ${QNN_SDK_ROOT}/lib/x86_64-linux-clang/libQnnSystem.so
```

9 Develop custom applications

This section explains how to develop custom C++ applications that integrate the embedded Neural Processing Unit (eNPU) using Qualcomm Neural Network (QNN) APIs and defined initialization, execution, and deinitialization flows. It describes how to use the `QnnInterface` to access QNN APIs and build production-ready, low-power AI (LPAI) applications that can run on a Linux host computer or execute directly on the target device.

The following table lists the QNN APIs supported by the LPAI backend:

Table : QNN APIs for LPAI backend

QNN API	Description
`QnnContext_createFromBinary`	Create a context from a stored binary.
`QnnBackend_create`	Initialize a backend library and create a backend handle.
`QnnGraph_setConfig`	Set/modify configuration options on an already created graph.
`QnnGraph_getProperty`	Retrieve a list of graph properties.
`QnnGraph_finalize`	Finalize the graph.
`QnnGraph_execute`	Synchronously execute a finalized graph.
`QnnContext_free`	Free the context and all associated graphs, operations, and tensors.
`QnnGraph_retrieve`	Retrieve a graph based on name.
`QnnContext_setConfig`	Set/modify configuration options on an already generated context.
`QnnBackend_free`	Free all resources associated with a backend handle.
`QnnBackend_getApiVersion`	Get the QNN API version.
`QnnBackend_getBuildId`	Get build ID for backend library.
`QnnProperty_hasCapability`	Query the capability of the backend.
`QnnProfile_create`	Create a handle to a profile object.
`QnnProfile_getEvents`	Get QNN profile events collected on the profile handle.
`QnnProfile_getSubEvents`	Get QNN profile event handles nested within the QNN profile event handle.
`QnnProfile_getEventData`	Query the data associated with QNN profile event.
`QnnProfile_getExtendedEventData`	Query the data associated with QNN profile extended event.
`QnnProfile_free`	Free memory associated with the profile handle. All associated `QnnProfile_EventId_t` event handles are implicitly freed.
`QnnDevice_getPlatformInfo`	Get the collection of devices and cores that a QNN backend can recognize and communicate with.
`QnnDevice_freePlatformInfo`	Free memory allocated during `QnnDevice_getPlatformInfo()`.
`QnnDevice_create`	Create a logical device to handle a subset of hardware resources available on the platform.
`QnnDevice_setConfig`	Free the created device and deallocate any resources allocated during device creation.
`QnnDevice_getInfo`	Get platform info associated with a device handle.
`QnnDevice_free`	Free the created device and deallocate any resources allocated during device creation.
`QnnMem_deRegister`	Deregisters a memory handle that's registered through `QnnMem_register` and invalidates `memHandle` for the specified backend handle.

QNN API	Description
QnnMem_register	Register existing memory to memory handle. Used to instruct QNN to use this memory directly.
QnnLog_create	Create a handle to a logger object. You may run this function prior to the QnnBackend_create().
QnnLog_setLogLevel	Set the log level for the supplied log handle.
QnnLog_free	Free the memory associated with the log handle.

The following table lists the QNN system APIs and their descriptions:

Table : QNN system APIs

QNN APIs	Description
QnnSystemContext_create	Create an instance of the QNN system context.
QnnSystemContext_free	Free the instance of the System Context object and clears any intermediate memory allocated and associated with a valid handle.
QnnSystemContext_getBinaryInfo	Get context info from the serialized binary buffer.

For a complete list of QNN APIs, see the QNN SDK documentation that's part of the SDK installation: <QNN_SDK_DIR>/docs/QNN/apirst/unabridged_api.html.

9.1 Access QNN APIs using the QnnInterface

The QnnInterface structure provides access to QNN APIs.

- It is an abstraction that combines all component APIs into a single, unified interface.
- It allows developers to work with a fixed set of API hooks through the interface instead of relying on dynamic symbol lookup using dlsym() calls.
- QNN APIs are exposed through to users through API providers.
- Available API providers can be queried using QnnInterface_getProviders().

```
const QnnInterface_t **providerlist = NULL;
uint32_t num_providers = 0;
QnnError_Error_t result =
    QnnInterface_getProviders(&providerlist, &num_providers);
```

9.2 Integrate the model with QNN APIs call flow

QNN model integration comprises the following three phases: initialization, execution, and deinitialization.

9.2.1 Initialization

1. Retrieve the LPAI BE interface and the QNN system interface.
2. Create handles for the LPAI BE and QNN system contexts.
3. Query the buffer alignment requirements.
4. Allocate memory for QNN context binary buffer.
5. Create the QNN context using the context binary buffer.
6. Retrieve the graph information and graph name.
7. Retrieve the graph using the graph name.
8. Allocate scratch and persistent memory.
9. Set the scratch and persistent memory configurations in the graph.
10. Set the client performance requirements and eNPU core affinity.
11. Set or modify the QNN client priority.
12. Finalize the graph.
13. Allocate input and output tensors.

The following figure shows the initialization call flow:

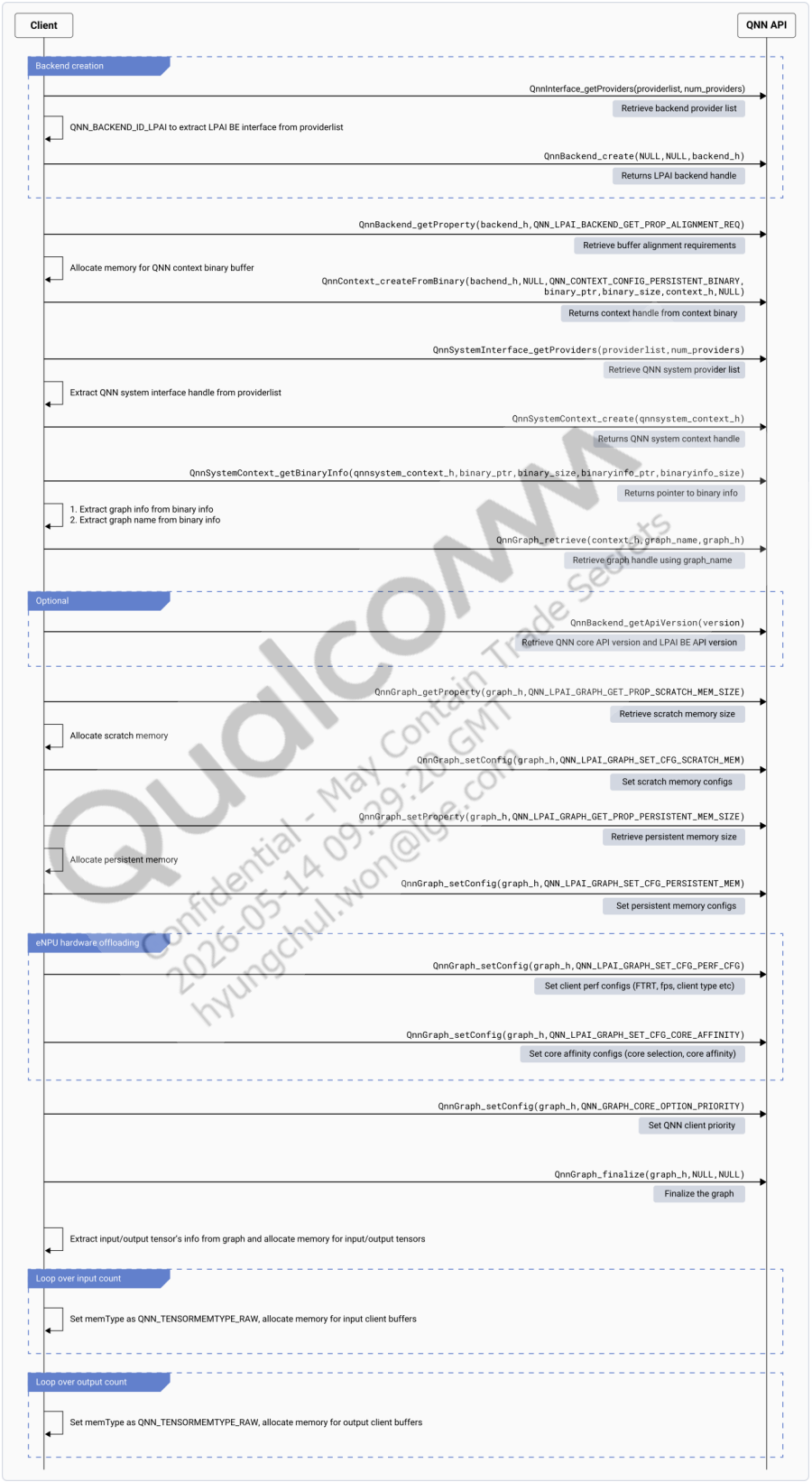


Figure : QNN initialization call flow

9.2.2 Execution

Copy the input data into the input client buffers. Use the `QnnGraph_execute()` API to run the finalized graph.

The following figure shows the execution call flow:

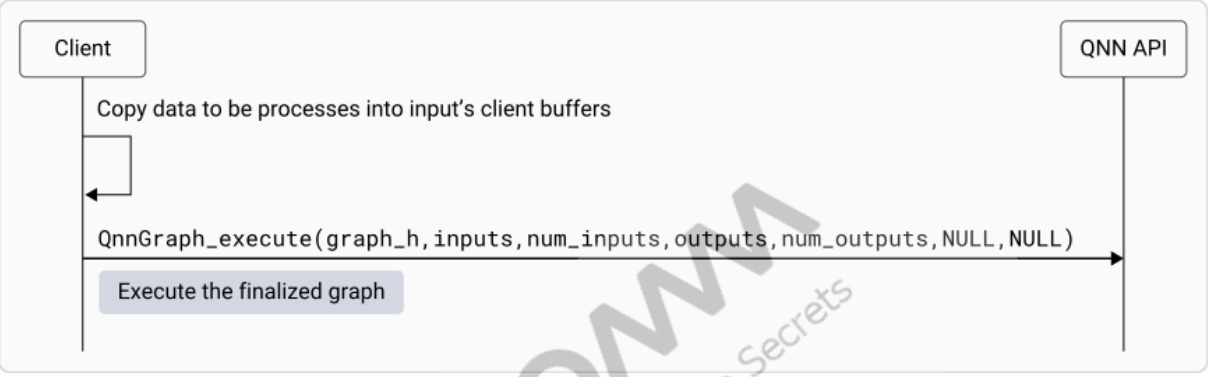


Figure : QNN execution call flow

9.2.3 Deinitialization

1. Release the QNN context handle.
2. Release the LPAI backend handle.
3. Release the QNN system context handle.
4. Free the scratch and persistent memory.
5. Free the input and output tensors and associated client buffers.

The following figure shows the deinitialization call flow:

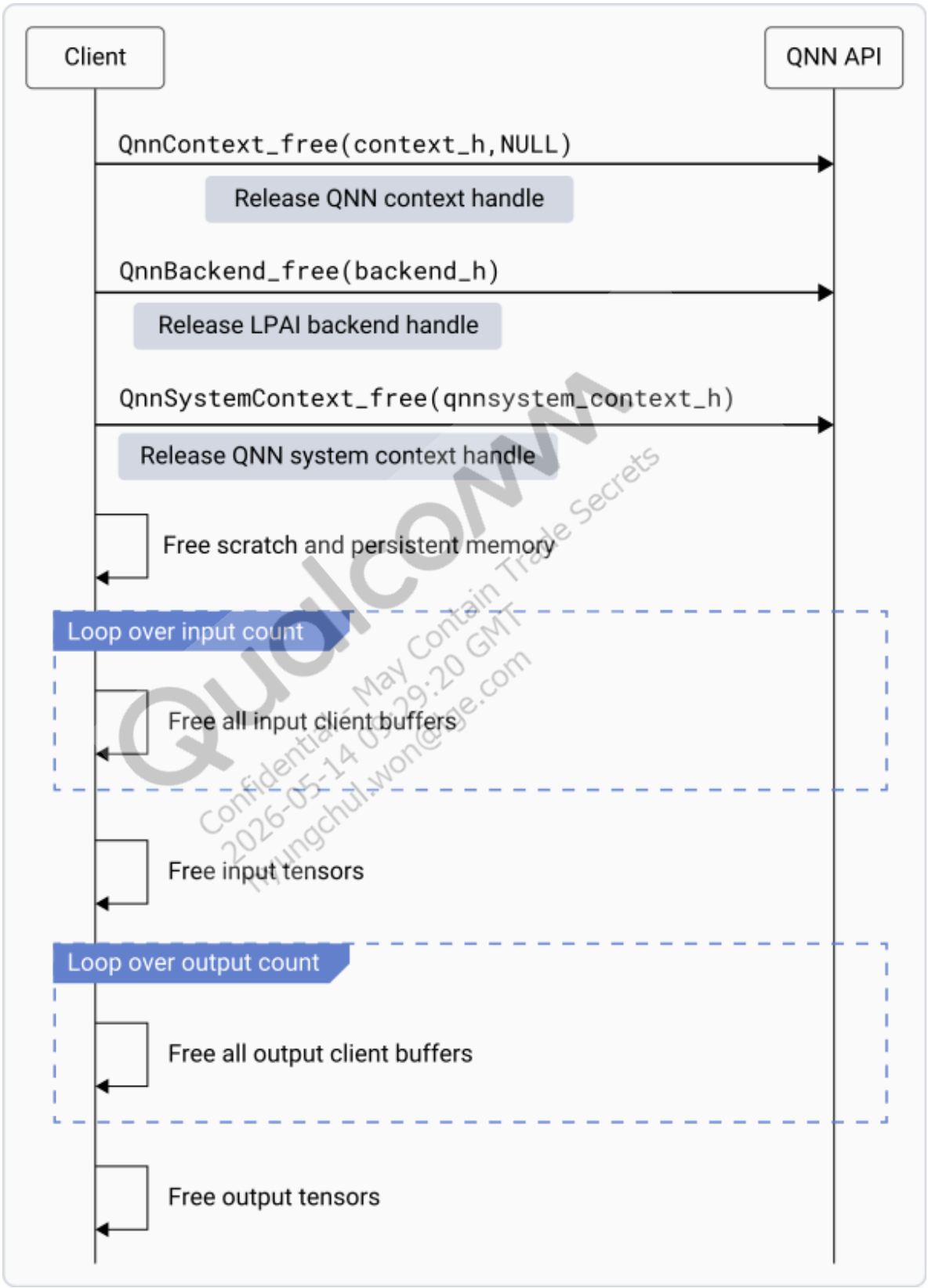


Figure : QNN deinitialization call flow

10 References

10.1 Related documents

Title	Number
Qualcomm Technologies, Inc.	
eNPU6 Design Guidelines	80-37209-5
Qualcomm AI Runtime (QAIRT) SDK	80-63442-10
SW6100P Linux Build Guide	80-93264-3

10.2 Acronyms and terms

Acronym or Term	Definition
ADB	Android Debug Bridge
aDSP	Audio digital signal processor
AIC	AI core (Qualcomm accelerator)
API	Application programming interface
BIN	Binary file format
CLANG	LLVM C/C++ compiler
CPU	Central processing unit
CPP	C++ source file
cDSP	Compute DSP
DDR	Double data rate memory
DSP	Digital signal processor
ECNS	Echo cancellation noise suppression
eNPU	Embedded neural processing unit
FPS	Frames per second
FTRT	Faster than real-time
GCC	GNU compiler collection
GLINK	High-speed inter-processor communication protocol
GPU	Graphics processing unit
GRU	Gated recurrent unit
HMX	Hexagon matrix extensions
HTA	Hardware tensor accelerator
HTP	Hexagon tensor processor
HVX	Hexagon vector extensions
IPC	Inter-processor communication
IR	Intermediate representation
JSON	JavaScript object notation
LE	Linux embedded

Acronym or Term	Definition
LLVM	Low-level virtual machine compiler framework
LPAI	Low-power AI
LPAIDSP	Low-power AI DSP
LPASS	Low-power subsystem
LPM	Low-power mode
LSTM	Long short-term memory
M55	ARM Cortex-M55 microcontroller (part of QSH)
MSVC	Microsoft Visual C++ compiler
NCHW / NHWC	Tensor layout formats
NDK	Android native development kit
ONNX	Open Neural Network Exchange
PD	Protection domain
QAIRT	Qualcomm AI runtime
QNN	Qualcomm neural network
QPM	Qualcomm package manager
QSH	Qualcomm sensing hub
SDK	Software development kit
TFLite	TensorFlow lite
ULPM	Ultra-low power mode
VENV	Virtual Python environment
VTCM	Vector tightly coupled memory

LEGAL INFORMATION

Your access to and use of this material, along with any documents, software, specifications, reference board files, drawings, diagnostics and other information contained herein (collectively this "Material"), is subject to your (including the corporation or other legal entity you represent, collectively "You" or "Your") acceptance of the terms and conditions ("Terms of Use") set forth below. If You do not agree to these Terms of Use, you may not use this Material and shall immediately destroy any copy thereof.

1) Legal Notice.

This Material is being made available to You solely for Your internal use with those products and service offerings of Qualcomm Technologies, Inc. ("**Qualcomm Technologies**"), its affiliates and/or licensors described in this Material, and shall not be used for any other purposes. If this Material is marked as "**Qualcomm Internal Use Only**", no license is granted to You herein, and You must immediately (a) destroy or return this Material to Qualcomm Technologies, and (b) report Your receipt of this Material to qualcomm.support@qti.qualcomm.com. This Material may not be altered, edited, or modified in any way without Qualcomm Technologies' prior written approval, nor may it be used for any machine learning or artificial intelligence development purpose which results, whether directly or indirectly, in the creation or development of an automated device, program, tool, algorithm, process, methodology, product and/or other output. Unauthorized use or disclosure of this Material or the information contained herein is strictly prohibited, and You agree to indemnify Qualcomm Technologies, its affiliates and licensors for any damages or losses suffered by Qualcomm Technologies, its affiliates and/or licensors for any such unauthorized uses or disclosures of this Material, in whole or part.

Qualcomm Technologies, its affiliates and/or licensors retain all rights and ownership in and to this Material. No license to any trademark, patent, copyright, mask work protection right or any other intellectual property right is either granted or implied by this Material or any information disclosed herein, including, but not limited to, any license to make, use, import or sell any product, service or technology offering embodying any of the information in this Material.

THIS MATERIAL IS BEING PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESSED, IMPLIED, STATUTORY OR OTHERWISE. TO THE MAXIMUM EXTENT PERMITTED BY LAW, QUALCOMM TECHNOLOGIES, ITS AFFILIATES AND/OR LICENSORS SPECIFICALLY DISCLAIM ALL WARRANTIES OF TITLE, MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, SATISFACTORY QUALITY, COMPLETENESS OR ACCURACY, AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MOREOVER, NEITHER QUALCOMM TECHNOLOGIES, NOR ANY OF ITS AFFILIATES AND/OR LICENSORS, SHALL BE LIABLE TO YOU OR ANY THIRD PARTY FOR ANY EXPENSES, LOSSES, USE, OR ACTIONS HOWSOEVER INCURRED OR UNDERTAKEN BY YOU IN RELIANCE ON THIS MATERIAL.

Certain product kits, tools and other items referenced in this Material may require You to accept additional terms and conditions before accessing or using those items.

Technical data specified in this Material may be subject to U.S. and other applicable export control laws. Transmission contrary to U.S. and any other applicable law is strictly prohibited.

Nothing in this Material is an offer to sell any of the components or devices referenced herein.

This Material is subject to change without further notification.

In the event of a conflict between these Terms of Use and the Website Terms of Use on www.qualcomm.com, the *Qualcomm Privacy Policy* referenced on www.qualcomm.com, or other legal statements or notices found on prior pages of the Material, these Terms of Use will control. In the event of a conflict between these Terms of Use and any other agreement (written or click-through, including, without limitation any non-disclosure agreement) executed by You and Qualcomm Technologies or a Qualcomm Technologies affiliate and/or licensor with respect to Your access to and use of this Material, the other agreement will control.

These Terms of Use shall be governed by and construed and enforced in accordance with the laws of the State of California, excluding the U.N. Convention on International Sale of Goods, without regard to conflict of laws principles. Any dispute, claim or controversy arising out of or relating to these Terms of Use, or the breach or validity hereof, shall be adjudicated only by a court of competent jurisdiction in the county of San Diego, State of California, and You hereby consent to the personal jurisdiction of such courts for that purpose.

2) Trademark and Product Attribution Statements.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the U.S. and/or elsewhere. The Bluetooth® word mark is a registered trademark owned by Bluetooth SIG, Inc. Other product and brand names referenced in this Material may be trademarks or registered trademarks of their respective owners.

Snapdragon and Qualcomm branded products referenced in this Material are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patented technologies are licensed by Qualcomm Incorporated.

THE DOCUMENTATION ACCOMPANYING THE MATERIALS AND/OR RELEVANT PRODUCTS AND SERVICE OFFERINGS MAY INCLUDE IMPORTANT USE LIMITATIONS. ANY DEVIATIONS FROM APPLICABLE USE LIMITATIONS MAY ADVERSELY IMPACT PERFORMANCE, DURABILITY, QUALITY OR SAFETY. YOU ASSUME ALL RISKS AND LIABILITIES ASSOCIATED WITH ANY DEVIATIONS.